

Digital Design & Computer Arch.

Lecture 15a: Dataflow & Superscalar Execution

Dr. Rahul Bera
Prof. Onur Mutlu

ETH Zürich
Spring 2026
17 April 2026

Roadmap for Today (and Past Few Lectures)

■ Microarchitecture Fundamentals

- ❑ Single-cycle Microarchitectures
- ❑ Multi-cycle Microarchitectures

■ Pipelining & Precise Exceptions

- ❑ Pipelining
- ❑ Pipelined Processor Design
 - Control & Data Dependence Handling
 - Precise Exceptions: State Maintenance & Recovery

Problem
Algorithm
Program/Language
System Software
SW/HW Interface
Micro-architecture
Logic
Devices
Electrons

■ Last+this week: Out-of-Order & Superscalar Execution

- ❑ Out-of-Order Execution
- ❑ Dataflow & Superscalar Execution
- ❑ Branch Prediction

Readings

■ This week

- ❑ Smith and Sohi, “The Microarchitecture of Superscalar Processors,” Proceedings of the IEEE, 1995
- ❑ H&H Chapters 7.8 and 7.9
- ❑ McFarling, “Combining Branch Predictors,” DEC WRL Technical Report, 1993.
- ❑ Optional: Kessler, “The Alpha 21264 Microprocessor,” IEEE Micro 1999.

Out-of-Order Execution (Restricted Dataflow) **Review & Wrap Up**

Recall: Enabling OoO Execution, Revisited

1. **Link** the consumer of a value to the producer
 - ❑ **Register renaming**: Associate a “tag” with each data value
2. **Buffer** instructions until they are ready
 - ❑ Insert instruction into **reservation stations** after renaming
3. Keep **track** of **readiness** of source values of an instruction
 - ❑ **Broadcast the “tag”** when the value is produced
 - ❑ Instructions **compare their “source tags”** to the broadcast tag
→ if match, source value becomes ready
4. When all source values of an instruction are ready, **dispatch** the instruction to functional unit (FU)
 - ❑ **Wakeup and select/schedule** the instruction

Recall: Summary of OOO Execution Concepts

- **Register renaming** eliminates false dependences, enables linking of producer to consumers (**link**)
- **Buffering in reservation stations** enables the pipeline to move for independent (ready) instructions (**buffer**)
- **Tag & value broadcast** enables communication (of readiness & value) between instructions (**track readiness**)
- **Wakeup and select** enables out-of-order instruction dispatch into functional units (**dispatch**)

Recall: OOO Execution: **Restricted Dataflow**

- An out-of-order engine dynamically builds the dataflow graph of a piece of the program
- The dataflow graph is limited to the **instruction window**
 - Instruction window: all decoded but not yet retired instructions

Recall: State of RAT and RS in Cycle 7

Slightly harder tasks for you:

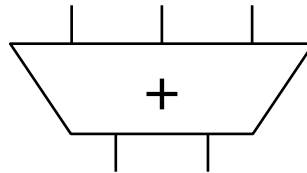
1. Draw the dataflow graph for the executing code
2. Provide the executing code in sequential order

Register	Valid	Tag	Value
R1	1		1
R2	1		2
R3	0	x	
R4	1		4
R5	0	d	
R6	1		6
R7	0	b	
R8	1		8
R9	1		9
R10	0	c	
R11	0	y	

Register Alias Table

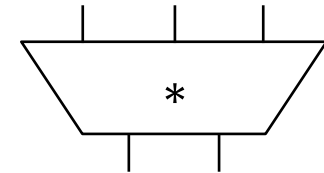
RS for ADD Unit

	Source 1			Source 2		
	V	Tag	Value	V	Tag	Value
a	0	x		1	~	4
b	1	~	2	1	~	6
c	1	~	8	1	~	9
d	0	a		0	y	



RS for MUL Unit

	Source 1			Source 2		
	V	Tag	Value	V	Tag	Value
x	1	~	1	1	~	2
y	0	b		0	c	
z						
t						



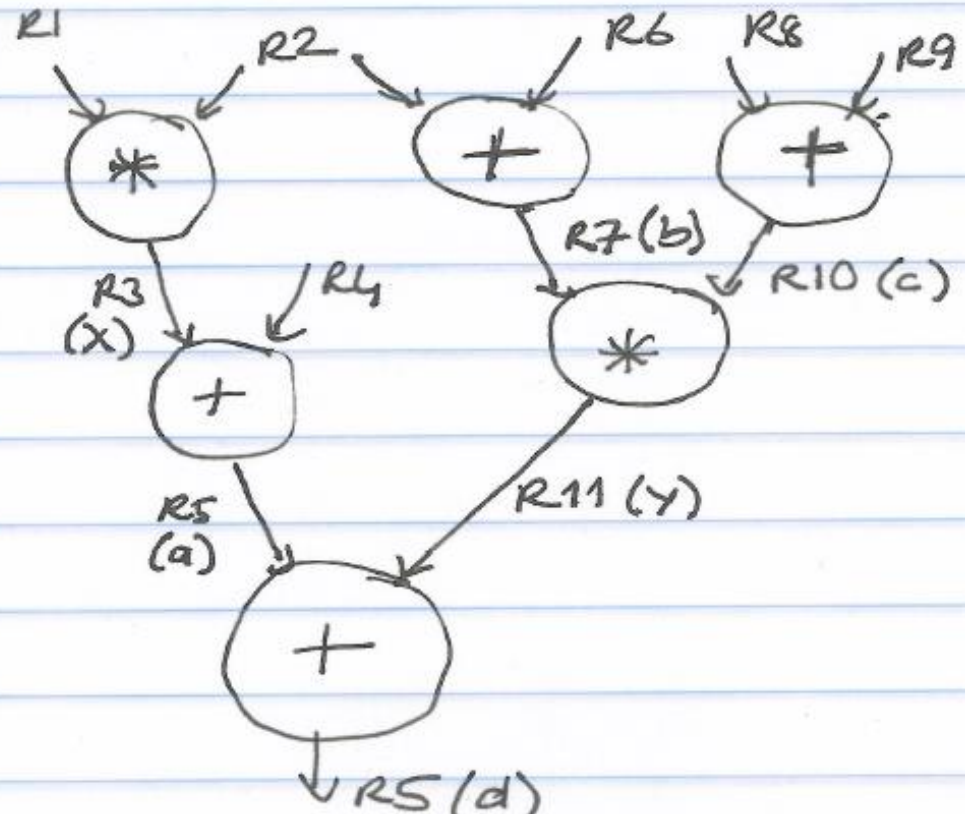
Recall: Reverse Engineered Dataflow Graph

MUL R1, R2 → R3 (x)
ADD R3, R4 → R5 (a)
ADD R2, R6 → R7 (b)
ADD R8, R9 → R10 (c)
MUL R7, R10 → R11 (y)
ADD R5, R11 → R5 (d)

Dataflow graph

Nodes: operations performed by the instruction

Arcs: tags in Tomasulo's algorithm

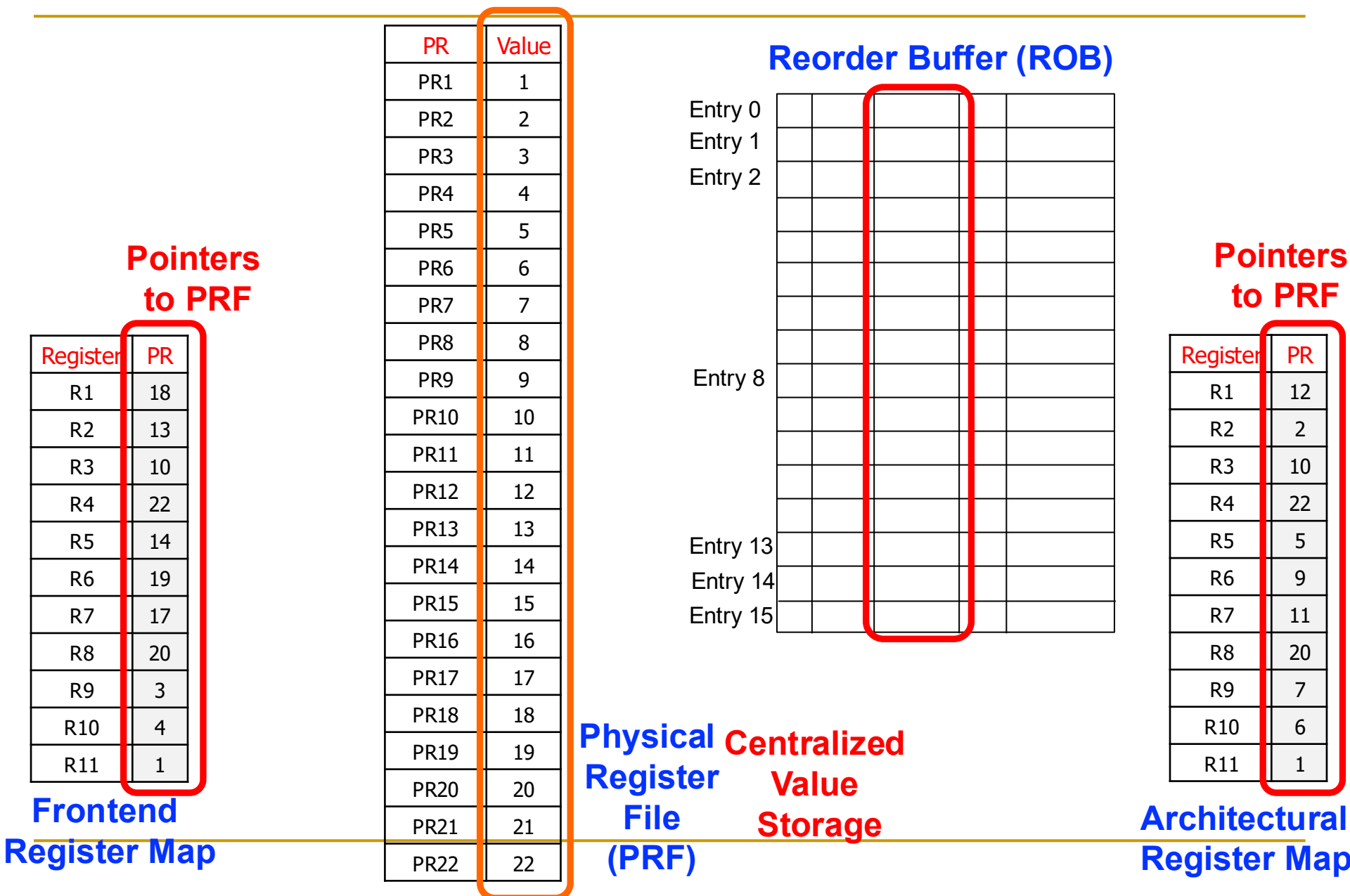


We can “easily” reverse-engineer the dataflow graph of the executing code!

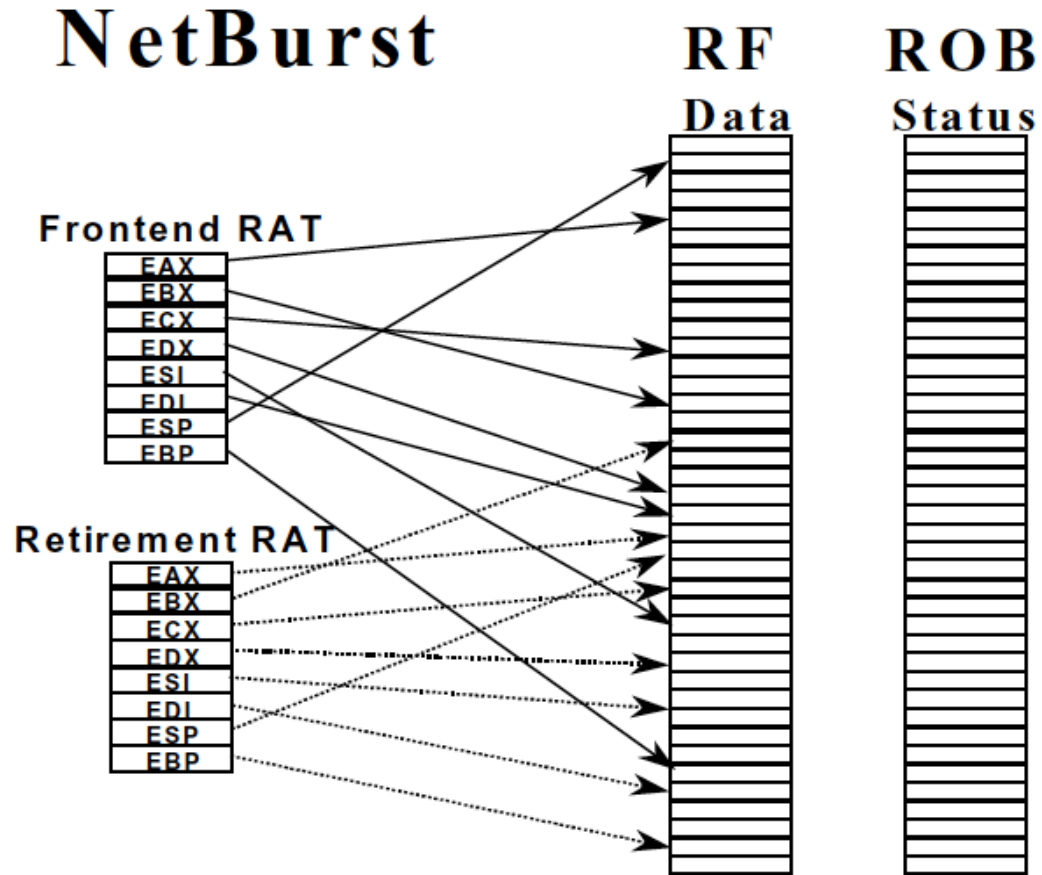
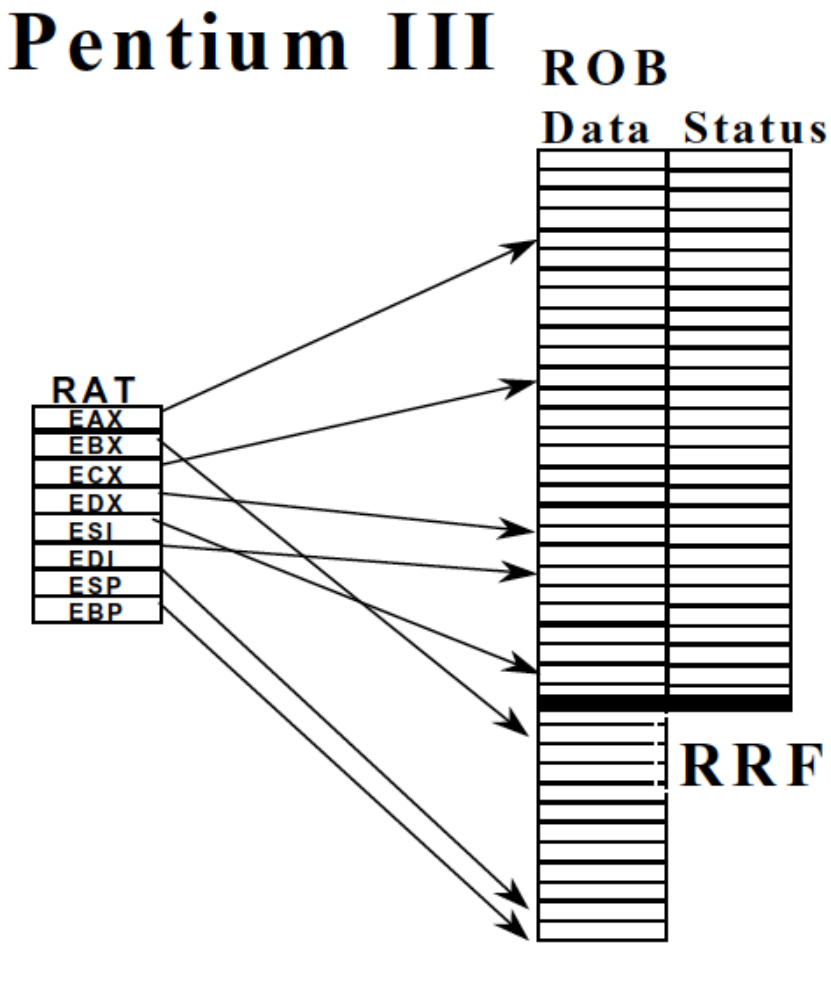
Recall: OoO Execution w/ Precise Exceptions

- Most modern processors use the following
 - **Reorder buffer** to support in-order retirement of instructions
 - A single register file (physical RF) to store **all registers**
 - Both speculative and architectural registers
 - INT and FP are still separate
 - Two register **maps** store **pointers** to the physical RF
 - Future/frontend register map → used for renaming
 - Architectural register map → used for maintaining precise state
- This design avoids value replication in RSs, ROB, etc.

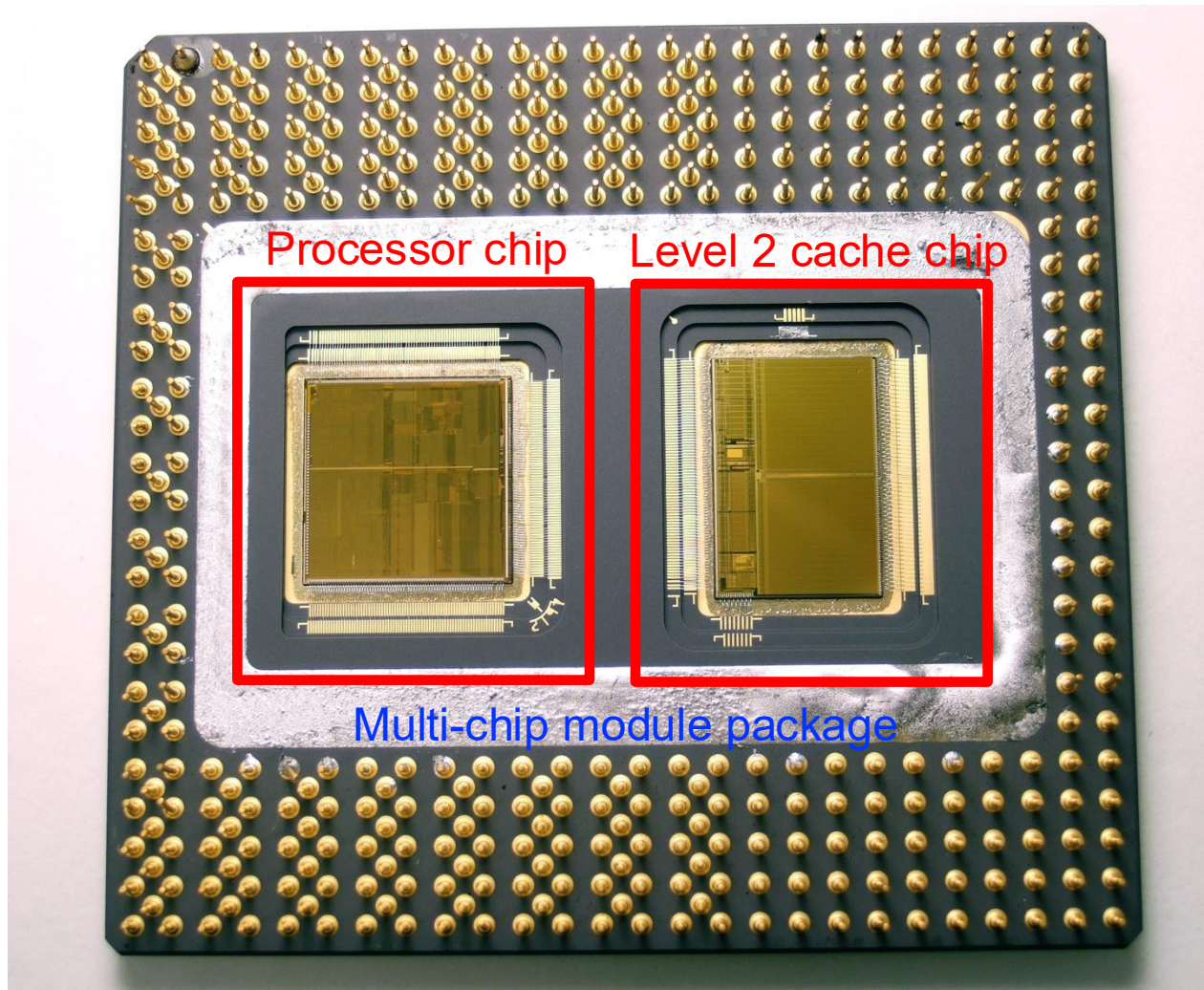
Recall: OoO Execution w/ Precise Exceptions (II)



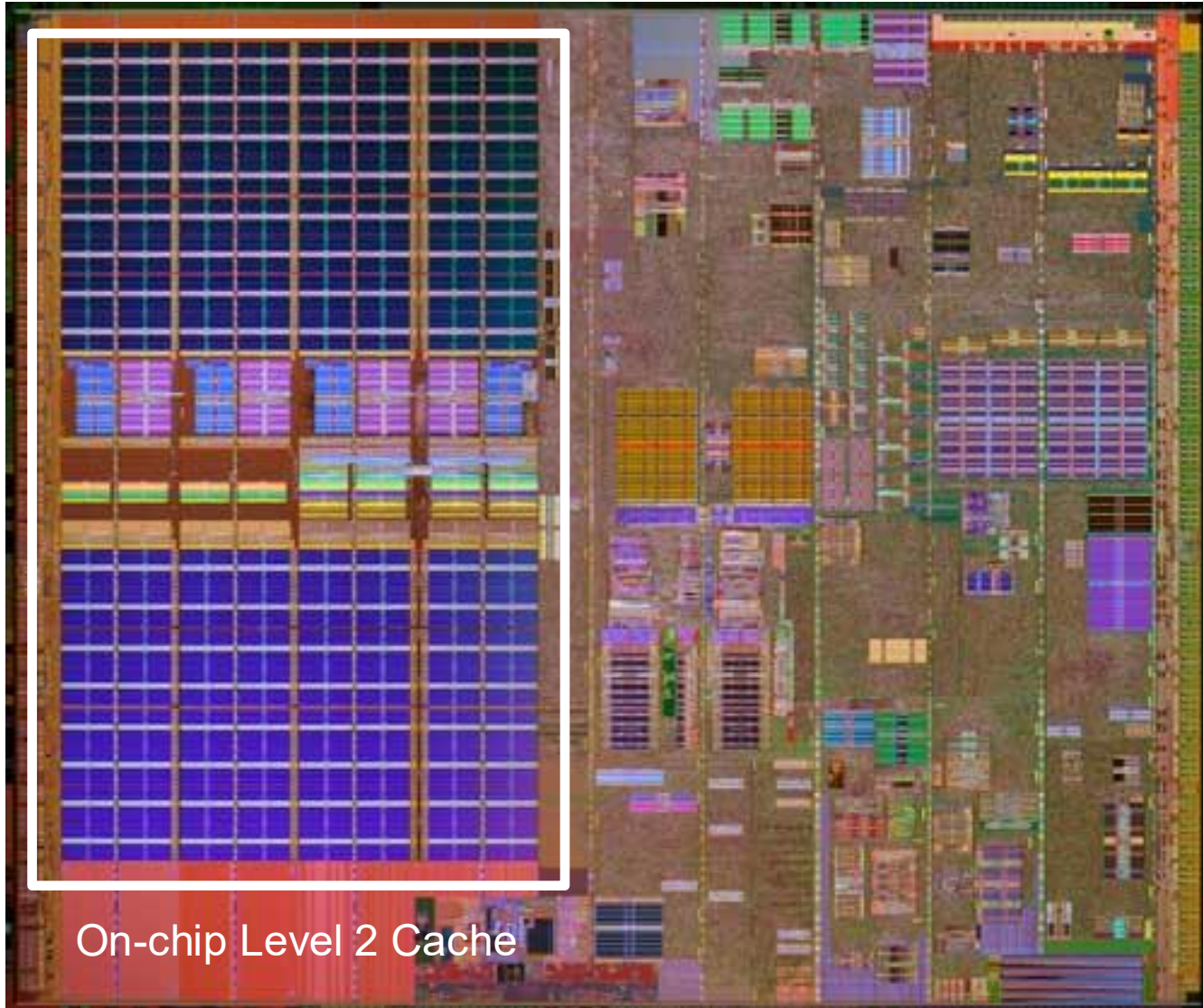
Recall: Examples from Modern Processors



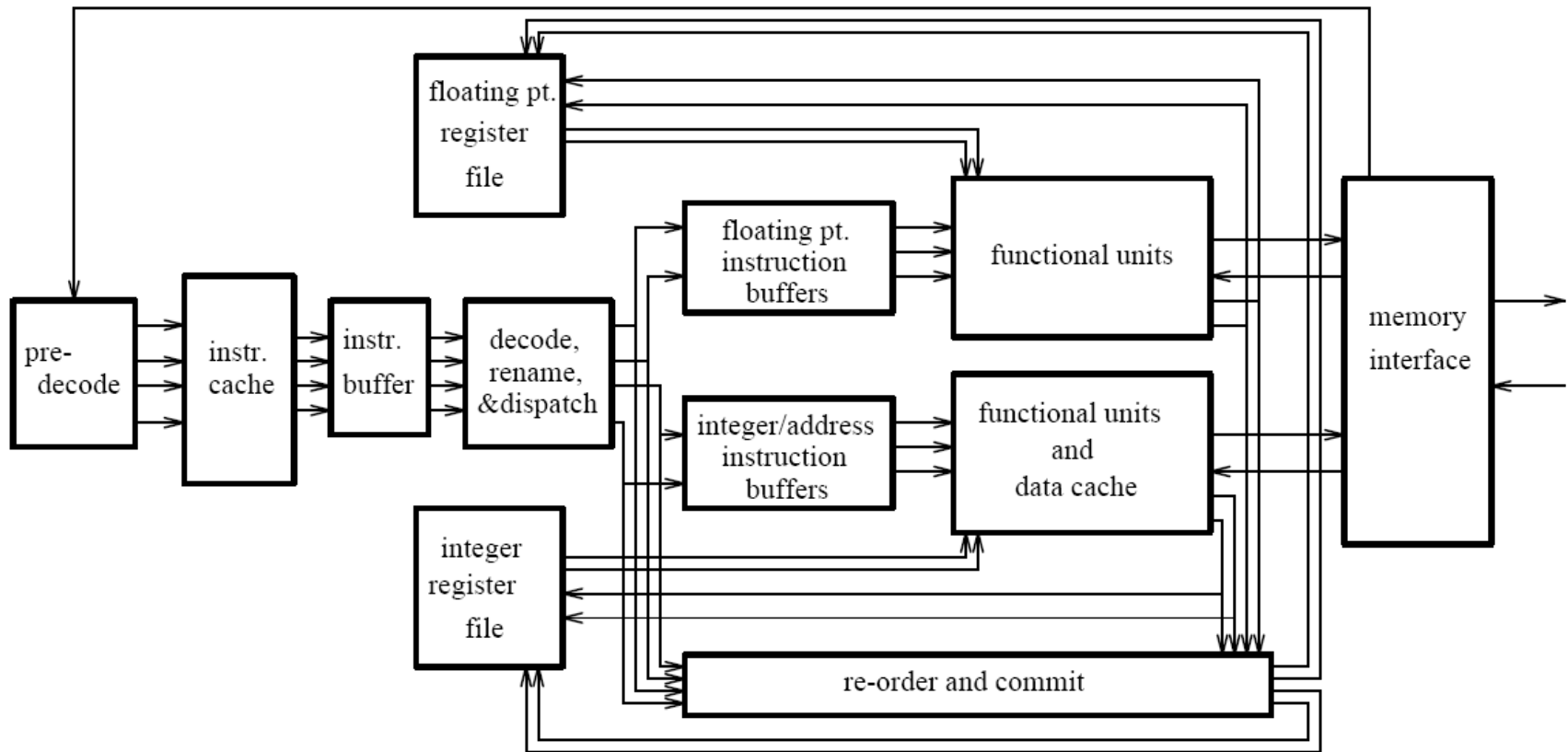
Intel Pentium Pro (1995)



Intel Pentium 4 (2000)



General Organization of an OOO Processor



- Smith and Sohi, “**The Microarchitecture of Superscalar Processors**,” Proc. IEEE, Dec. 1995.

A Modern OoO Design: Intel Pentium 4

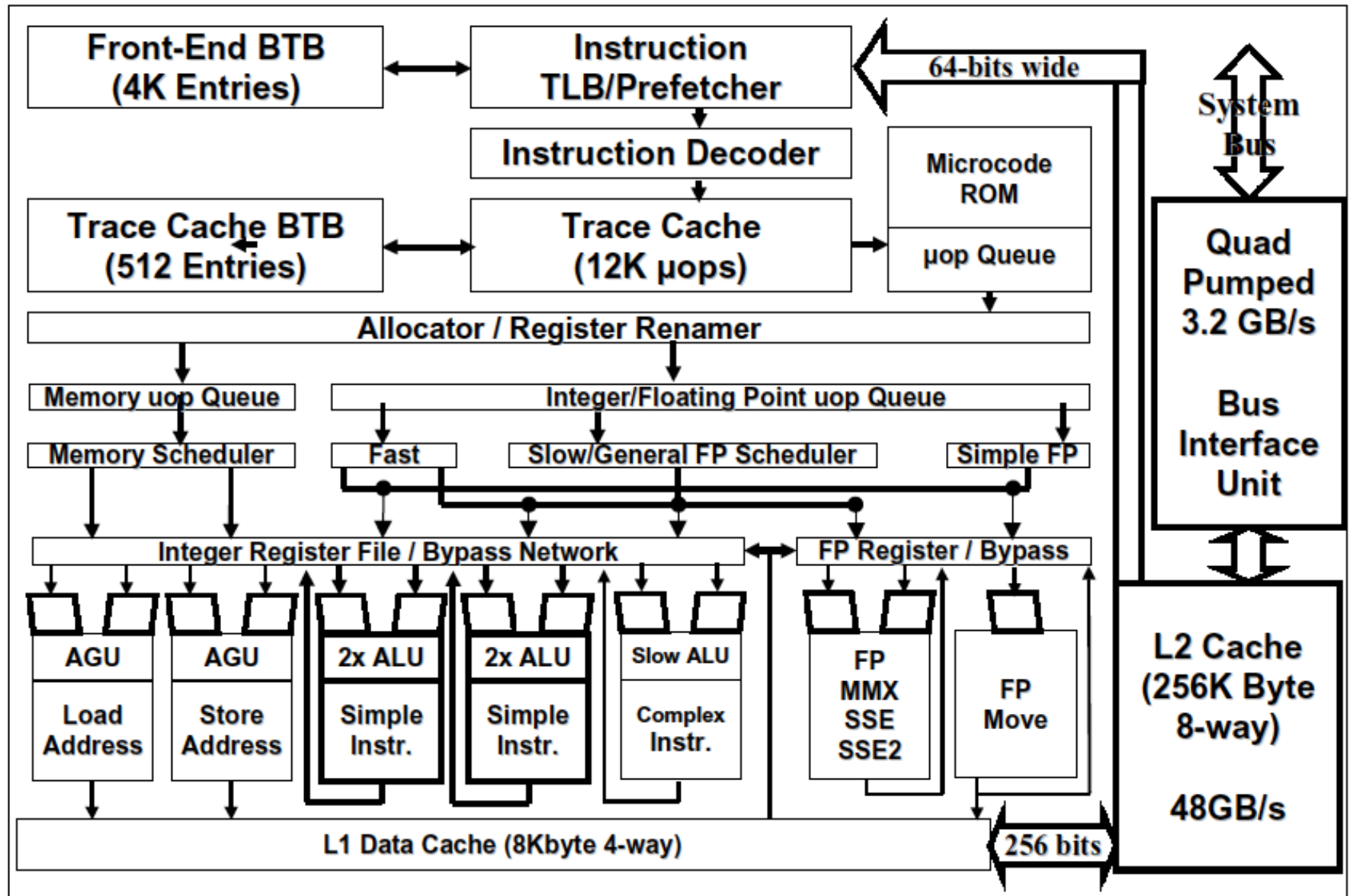
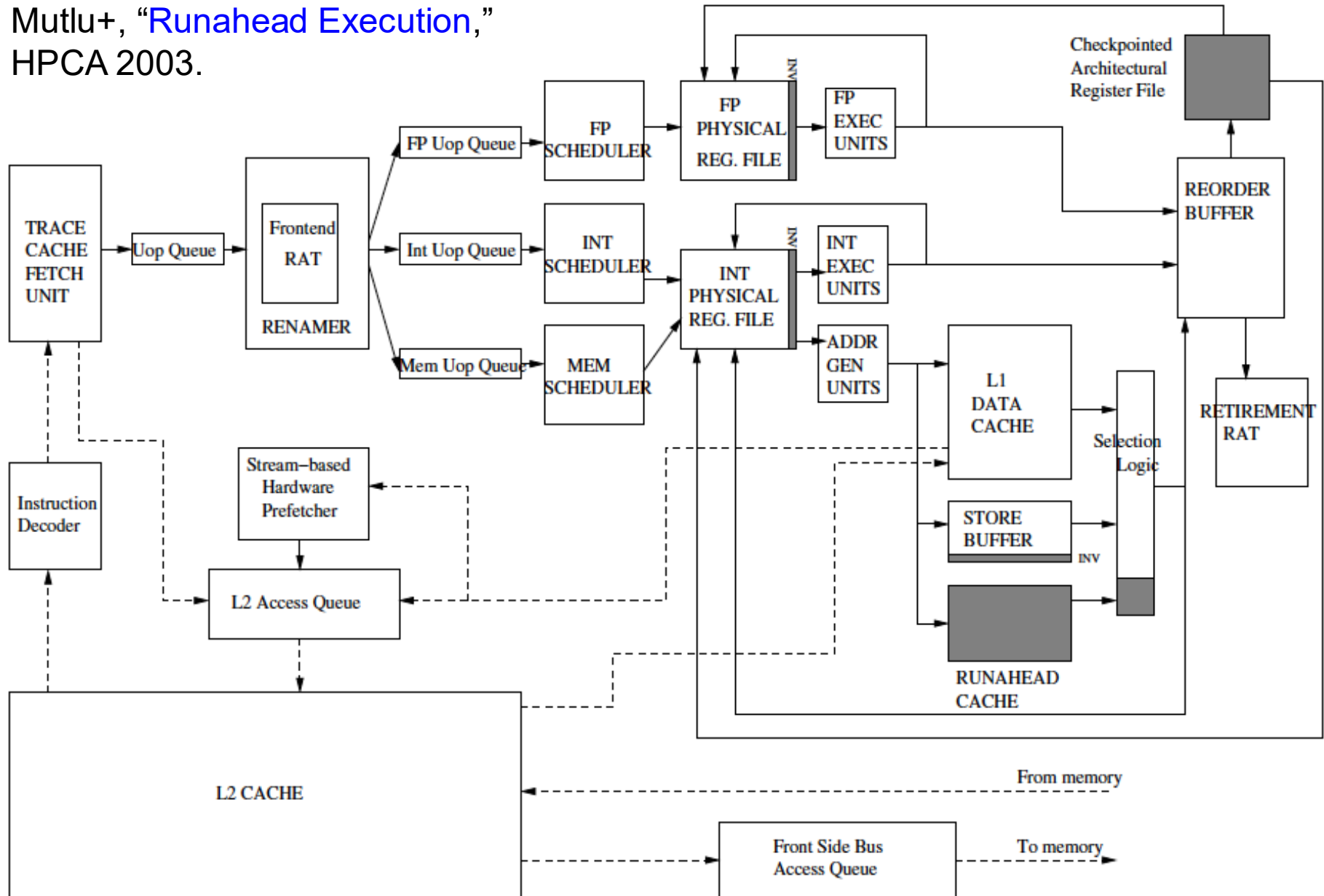


Figure 4: Pentium[®] 4 processor microarchitecture

Intel Pentium 4 Simplified

Mutlu+, “Runahead Execution,”
HPCA 2003.



Alpha 21264

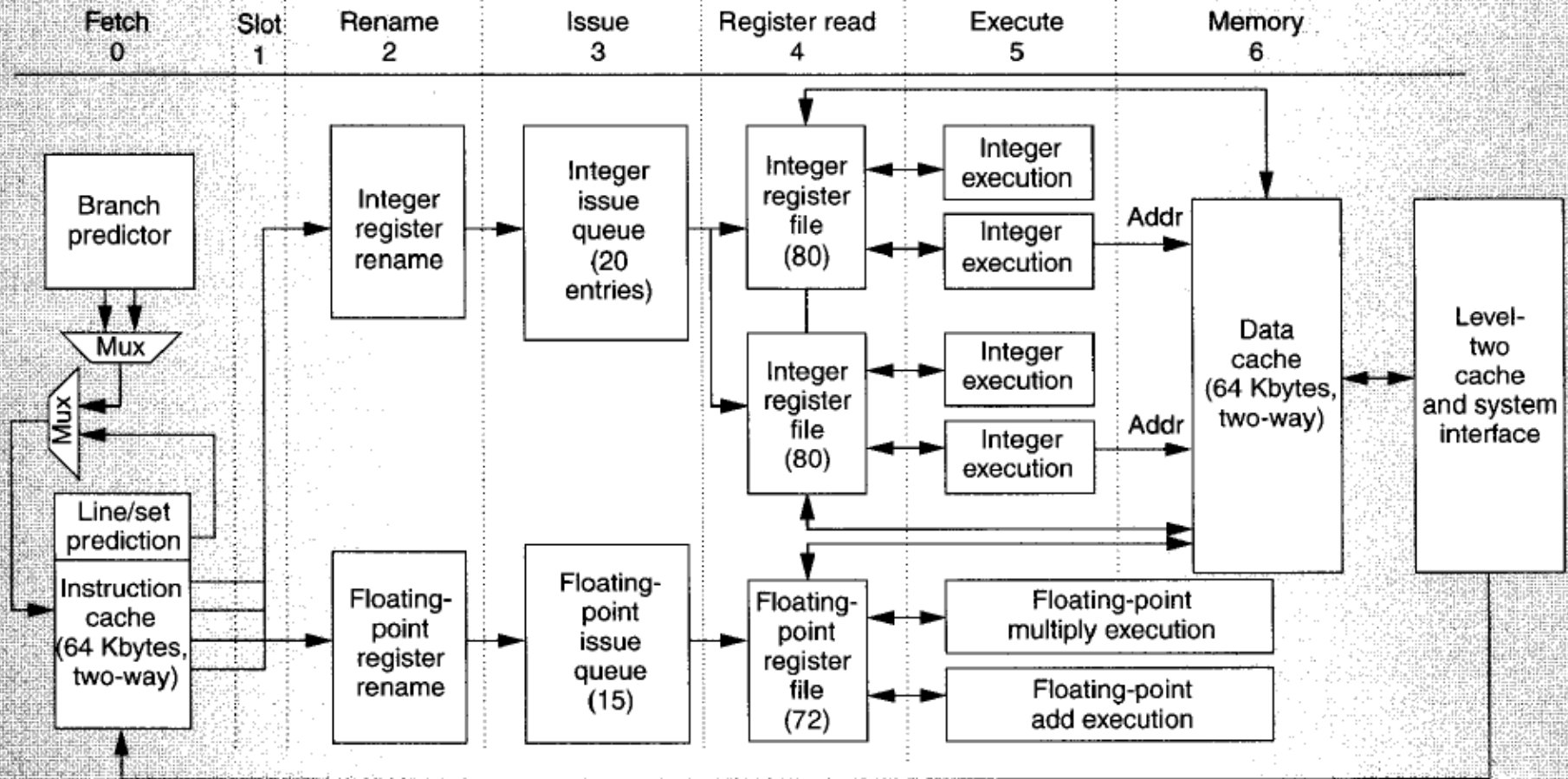
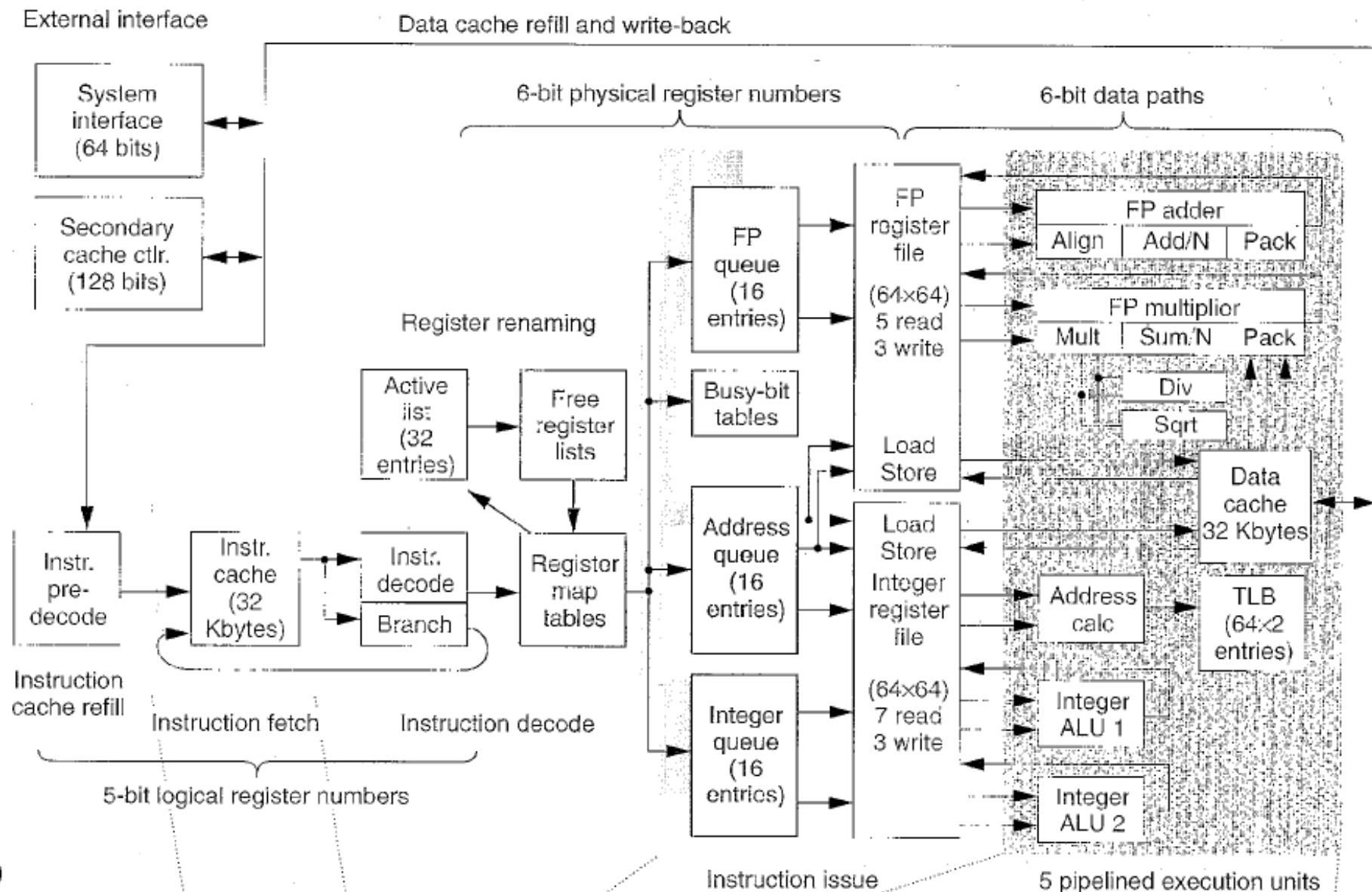


Figure 2. Stages of the Alpha 21264 instruction pipeline.

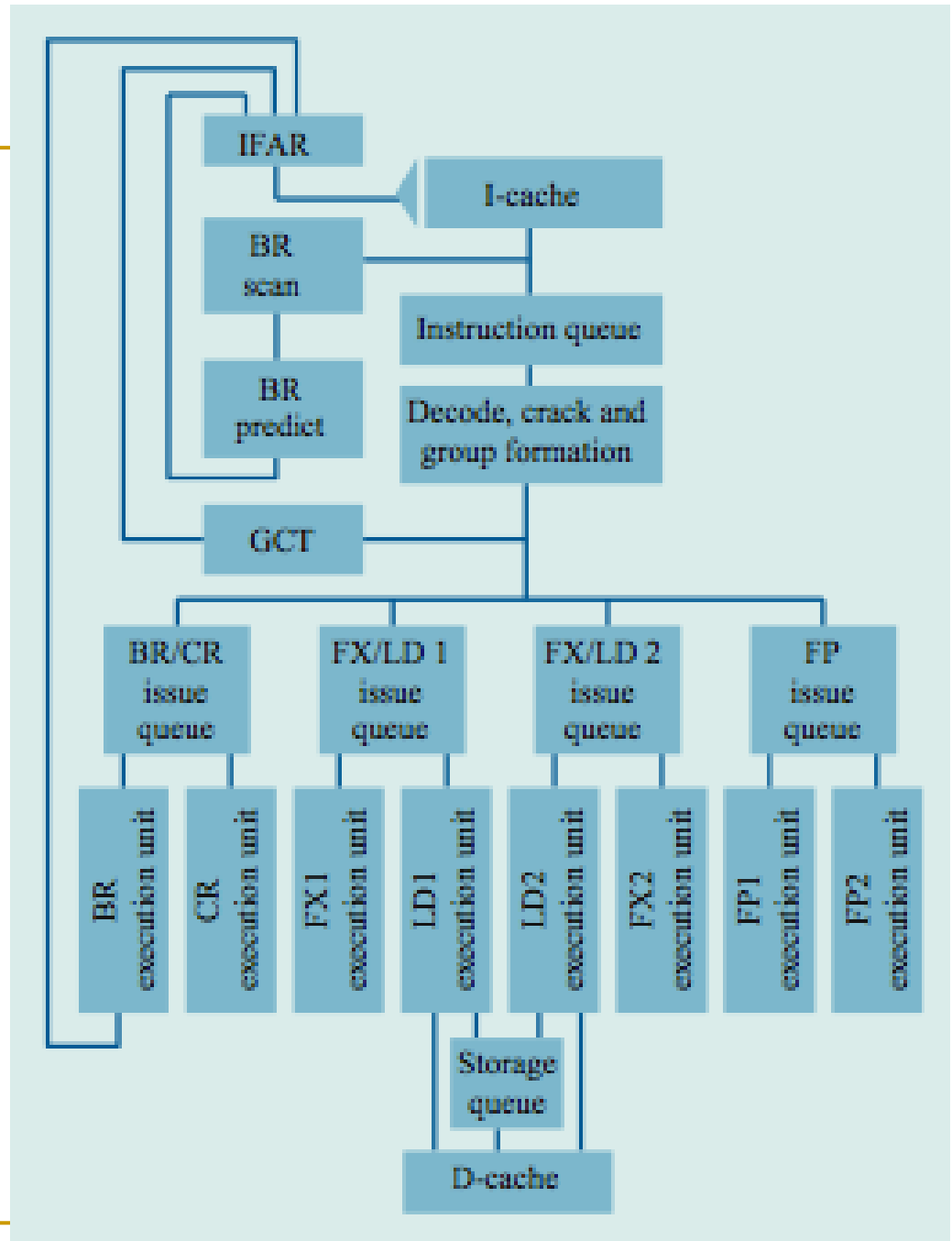
MIPS R10000



(a)

IBM POWER4

- Tendler et al.,
“POWER4 system
microarchitecture,”
IBM J R&D, 2002.



IBM POWER4

- 2 cores, out-of-order execution
- 100-entry instruction window in each core
- 8-wide instruction fetch, issue, execute
- Large, local+global hybrid branch predictor
- 1.5MB, 8-way L2 cache
- Aggressive stream based prefetching

IBM POWER5

- Kalla et al., “IBM Power5 Chip: A Dual-Core Multithreaded Processor,” IEEE Micro 2004.

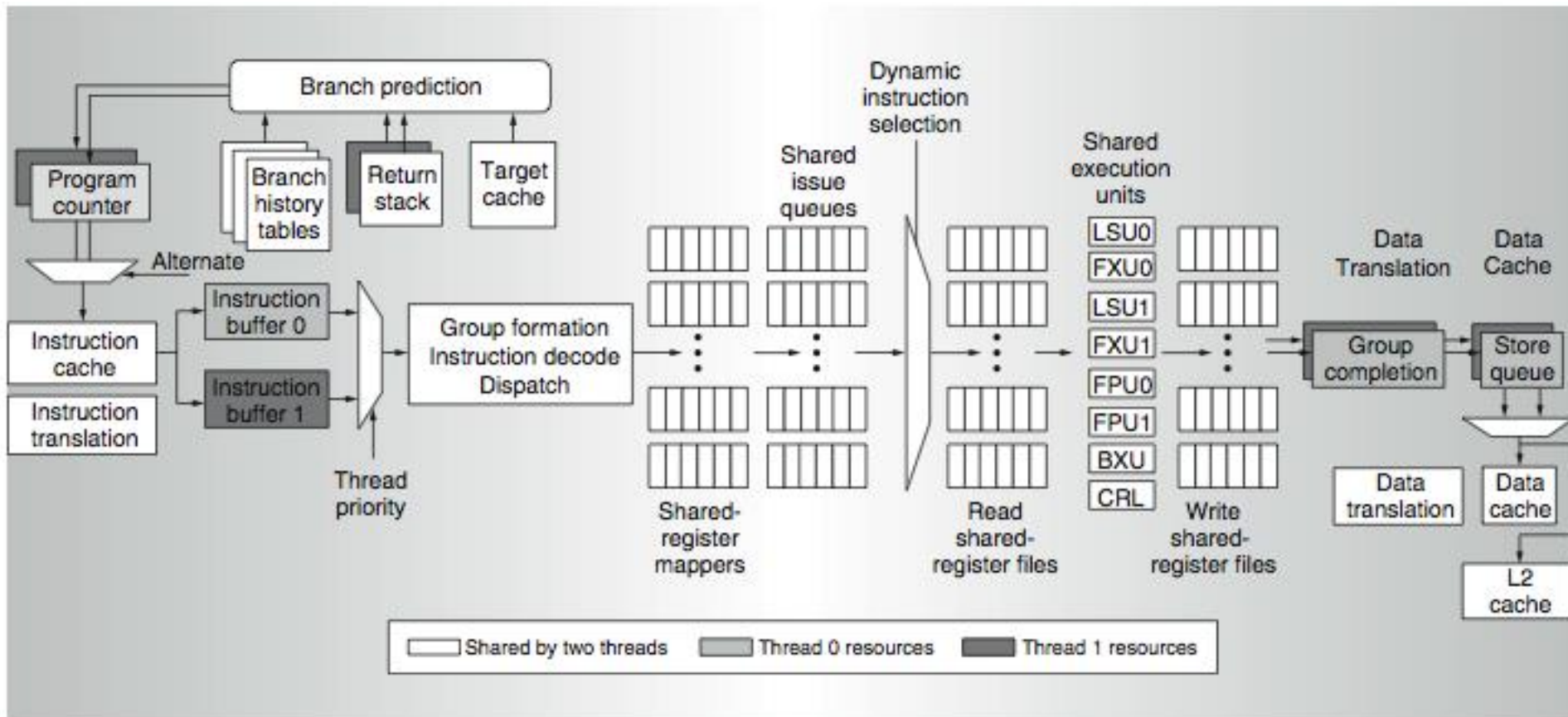
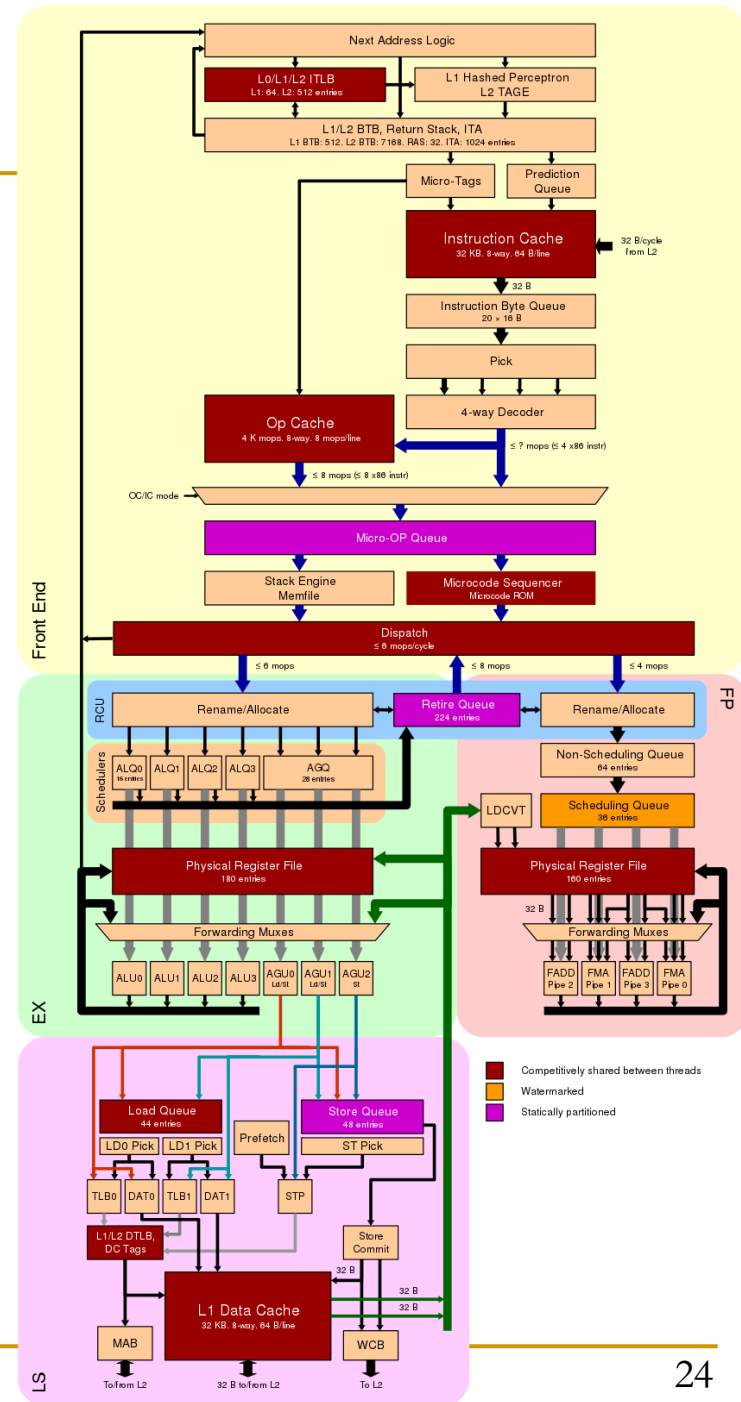
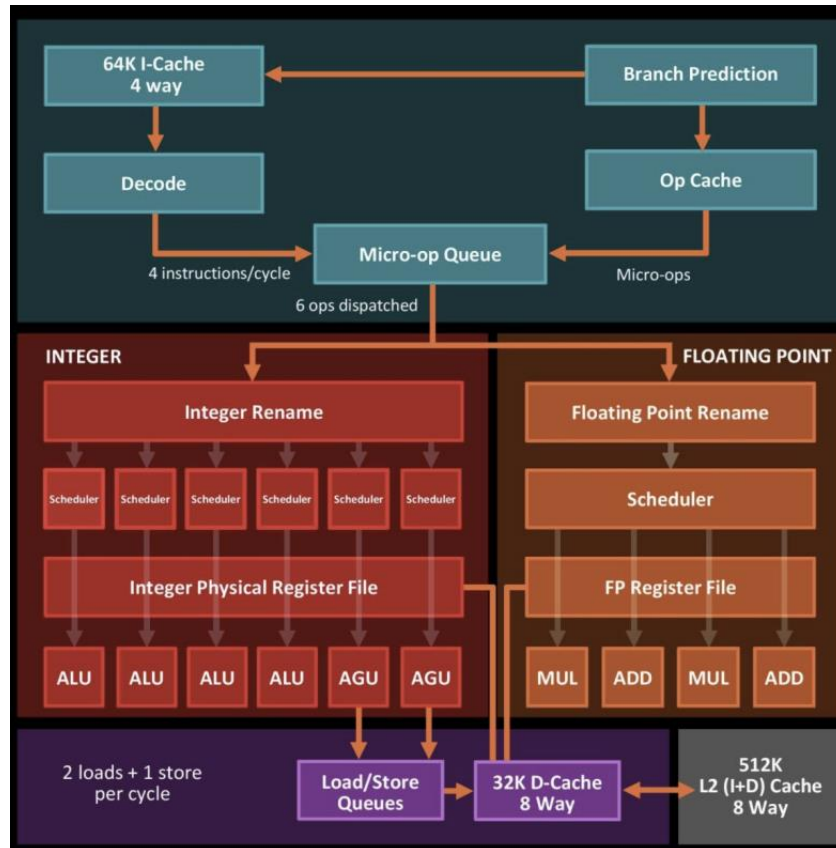
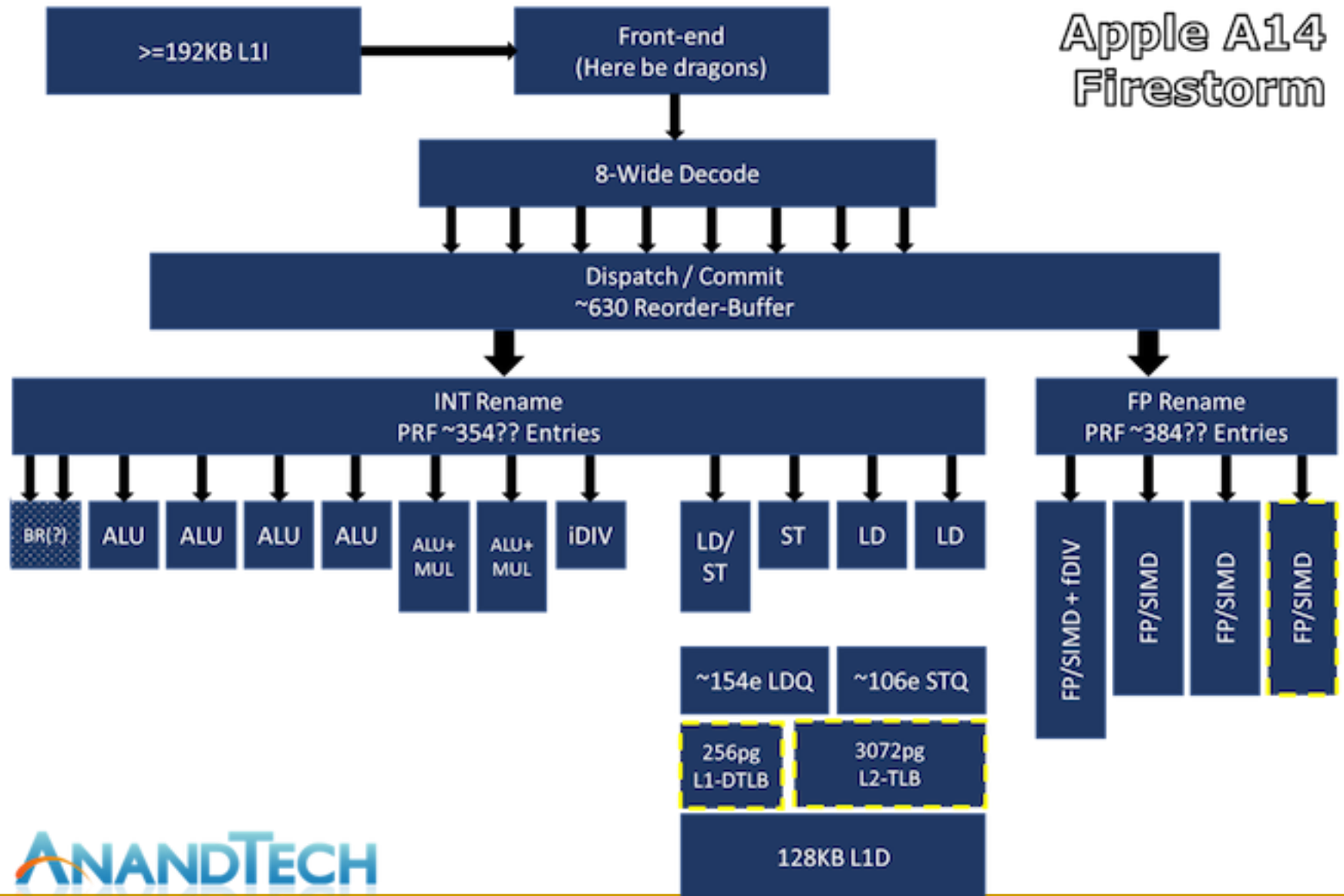


Figure 4. Power5 instruction data flow (BXU = branch execution unit and CRL = condition register logical execution unit).

AMD Zen/Zen2? (2019)



Apple M1 Firestorm? (2020)



Questions to Ponder

- Why is OoO execution beneficial?
 - **Latency tolerance**: OoO execution tolerates the latency of multi-cycle operations by executing independent operations concurrently
 - What if all operations take a single cycle?

- What if an instruction takes 1000 cycles?
 - How large of an instruction window do we need to continue decoding without stalling?
 - How many cycles of latency can OoO tolerate?
 - **What limits the latency tolerance scalability of Tomasulo's algorithm?**
 - **Instruction window size**: how many decoded but not yet retired instructions you can keep in the machine

Out-of-Order Execution Tradeoffs

■ Advantages

- **Latency tolerance**: Allows independent instructions to execute and complete in the presence of long-latency operations
 - Higher performance than in-order execution
- **Irregular parallelism**: Dynamically finds and exploits parallel operations in a program
 - Difficult to find/exploit such parallelism statically in irregular code

■ Disadvantages

- **Higher complexity**
 - Potentially lengthens critical path delay → clock cycle time
- **More hardware** resources needed

■ Recall: Execution time of an entire program

- **{# of instructions} x {Average CPI} x {clock cycle time}**

Lecture on Load-Store Handling in OoO

Store-Load Forwarding Complexity

- Content Addressable Search (based on Load Address)
- Range Search (based on Address and Size of both the Load and earlier Stores)
- Age-Based Search (for last written values)
- Load data can come from a combination of multiple places
 - One or more stores in the Store Buffer (SQ)
 - Memory/cache

69



22:38 / 24:03



Digital Design & Comp. Arch - Lecture 15b: Load-Store Handling in Out-of-Order Execution (Spring'23)



Onur Mutlu Lectures
32.5K subscribers



Subscribed



8



Share



Download



Clip



Save



<https://www.youtube.com/watch?v=UZAjQXUwzT4&list=PL5Q2soXY2Zi-ElmKxYYY1SZuGiOAOBKaf&index=19>

Other Approaches to Concurrency (or Instruction Level Parallelism)

Approaches to (Instruction-Level) Concurrency

- Pipelining
- Fine-Grained Multithreading
- Out-of-order Execution
- Dataflow (at the ISA level)
- Superscalar Execution
- VLIW
- SIMD Processing (Vector and array processors, GPUs)
- Decoupled Access Execute
- Systolic Arrays

Review: Data Flow: Exploiting Irregular Parallelism

Recall: OOO Execution: **Restricted Dataflow**

- An out-of-order engine dynamically builds the dataflow graph of a piece of the program
- The dataflow graph is limited to the **instruction window**
 - Instruction window: all decoded but not yet retired instructions

Recall: State of RAT and RS in Cycle 7

Slightly harder tasks for you:

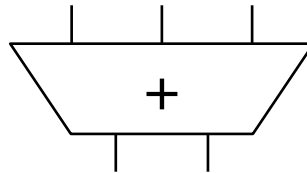
1. Draw the dataflow graph for the executing code
2. Provide the executing code in sequential order

Register	Valid	Tag	Value
R1	1		1
R2	1		2
R3	0	x	
R4	1		4
R5	0	d	
R6	1		6
R7	0	b	
R8	1		8
R9	1		9
R10	0	c	
R11	0	y	

Register Alias Table

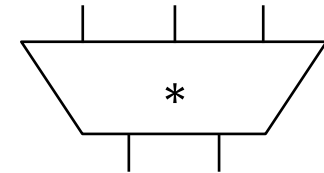
RS for ADD Unit

	Source 1			Source 2		
	V	Tag	Value	V	Tag	Value
a	0	x		1	~	4
b	1	~	2	1	~	6
c	1	~	8	1	~	9
d	0	a		0	y	



RS for MUL Unit

	Source 1			Source 2		
	V	Tag	Value	V	Tag	Value
x	1	~	1	1	~	2
y	0	b		0	c	
z						
t						



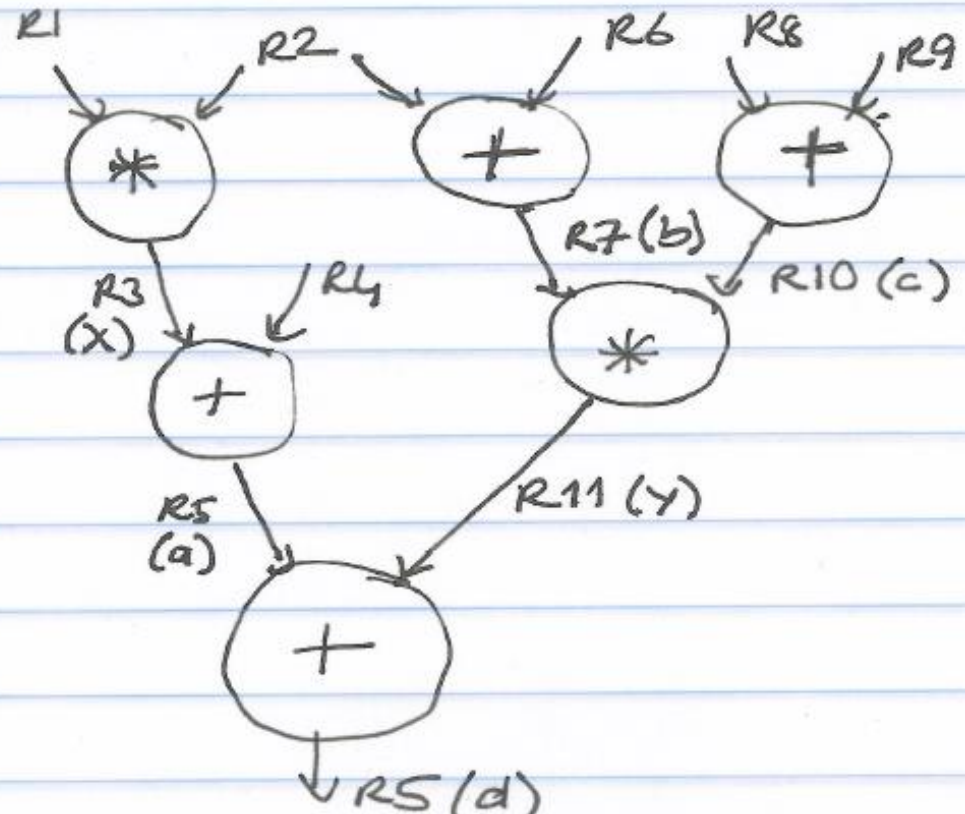
Recall: Reverse Engineered Dataflow Graph

MUL R1, R2 → R3 (x)
ADD R3, R4 → R5 (a)
ADD R2, R6 → R7 (b)
ADD R8, R9 → R10 (c)
MUL R7, R10 → R11 (y)
ADD R5, R11 → R5 (d)

Dataflow graph

Nodes: operations performed by the instruction

Arcs: tags in Tomasulo's algorithm



We can “easily” reverse-engineer the dataflow graph of the executing code!

Data Flow Summary

- Availability of data determines order of execution
- A data flow node fires when its sources are ready
- Programs represented as data flow graphs (of nodes)
- Data Flow at the ISA level has **not** been (as) successful
- **Data Flow** implementations at the **microarchitecture** level (while preserving von Neumann model semantics) have been **very successful**
 - Out of order execution is the prime example
- Data Flow mapping of programs to reconfigurable hardware substrates (FPGAs) has also been **successful** (recently)

Recall: ISA-level Tradeoff: Program Counter

- Do we want a Program Counter (PC or IP) in the ISA?
 - Yes: Control-driven, sequential execution
 - An instruction is executed when the PC points to it
 - PC automatically changes sequentially (except for control flow instructions) → sequential
 - No: Data-driven, parallel execution
 - An instruction is executed when all its operand values are available → dataflow
- Tradeoffs: MANY high-level ones
 - Ease of programming (for average programmers)?
 - Ease of compilation?
 - Performance: Extraction of parallelism?
 - Hardware complexity?

ISA-Level Data Flow Tradeoffs

■ Advantages

- Very good at exploiting irregular parallelism
 - Only real dependences constrain processing
 - More parallelism can be exposed than Von Neumann model

■ Disadvantages

- No precise state semantics
 - Debugging very difficult
 - Interrupt/exception handling is very difficult
- Large hardware overhead (tag matching, data/tag storage)
- Too much parallelism? (Parallelism control needed)
- How to enable mutable data structures
- ...

Recall: ISA vs. Microarchitecture Level Tradeoff

- A similar tradeoff (control vs. data-driven execution) can be made at the microarchitecture level
- ISA: Specifies how the **programmer sees** the instructions to be executed
 - Programmer sees a sequential, control-flow execution order vs.
 - Programmer sees a dataflow execution order
- Microarchitecture: How the **underlying implementation actually executes** instructions
 - Microarchitecture can execute instructions in any order as long as it obeys the semantics specified by the ISA when making the instruction results visible to software
 - Programmer should see the order specified by the ISA

Readings & Lectures on Data Flow Model

- Dennis and Misunas, "A preliminary architecture for a basic data-flow processor," ISCA 1974.
- Gurd et al., "The Manchester prototype dataflow computer," CACM 1985.
- More detailed Lecture Video & Slides on DataFlow:
 - <http://www.youtube.com/watch?v=D2uue7izU2c>
 - <http://www.ece.cmu.edu/~ece740/f13/lib/exe/fetch.php?media=onur-740-fall13-module5.2.1-dataflow-part1.ppt>

Lecture Video on Dataflow

Loops and Function Calls Summary

Figure 11. Interface for a procedure call. On the left a call of procedure P whose graph is on the right. P has one parameter and one return value. The actual parameter receives a new tag and is sent to the input node of P and concurrently a token containing address A is sent to the output node. This SEND-TO-DESTINATION node transmits the other input token to a node of which the address is contained in the first token. The effect is that, when the return value of the procedure becomes available, the output node sends the result to node A, which returns the tag belonging to the calling expression.

Figure 12. An implementation of a loop using tagged values. At the start of the loop a new tag tree is allocated. Values belonging to consecutive iterations receive consecutive tags within the tree. The tag from before the loop is inserted on tokens that exit from the loop.

Chalkboard:

$$S + A[j] \times B[j]$$
$$\langle f, 0, 3, L, \text{true} \rangle$$
$$\langle f, 0, 3, R, \text{true} \rangle$$

Video Player:

42:27 / 1:25:00

Carnegie Mellon - Parallel Computer Architecture 2012-Onur Mutlu - Lec 22 - Dataflow I

3,627 views • Apr 21, 2013

24 0 SHARE SAVE ...

Carnegie Mellon Computer Architecture
1.79K subscribers

SUBSCRIBED

Approaches to (Instruction-Level) Concurrency

- Pipelining
- Fine-Grained Multithreading
- Out-of-order Execution
- Dataflow (at the ISA level)
- Superscalar Execution
- VLIW
- SIMD Processing (Vector and array processors, GPUs)
- Decoupled Access Execute
- Systolic Arrays

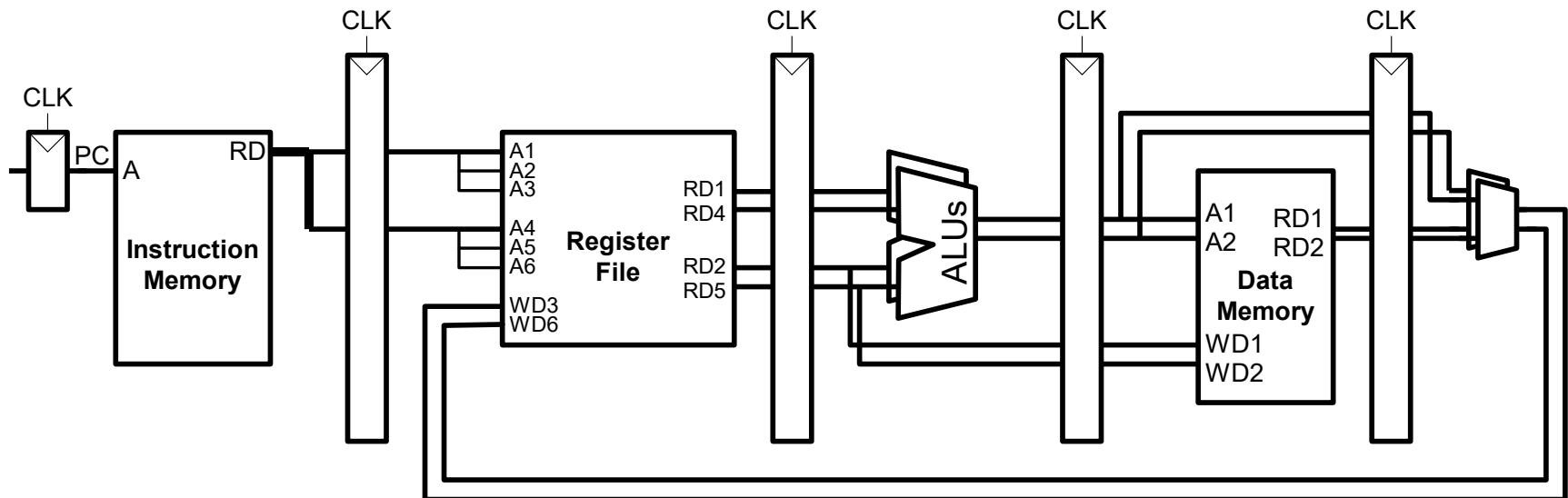
Superscalar Execution

Superscalar Execution

- Idea: Fetch, decode, execute, retire **multiple instructions per cycle**
 - N-wide superscalar → N instructions per cycle
- Need to add the hardware resources for doing so
- **Hardware** performs the **dependence checking between concurrently-fetched instructions**
- **Superscalar** execution and **out-of-order** execution are **orthogonal concepts**
 - Can have all four combinations of processors:
[in-order, out-of-order] x [scalar, superscalar]

In-Order Superscalar Processor Example

- Multiple copies of datapath: Can fetch/decode/execute multiple instructions per cycle
- Dependences make it tricky to dispatch multiple instructions in the same cycle
 - Need dependence detection between concurrently-fetched instructions

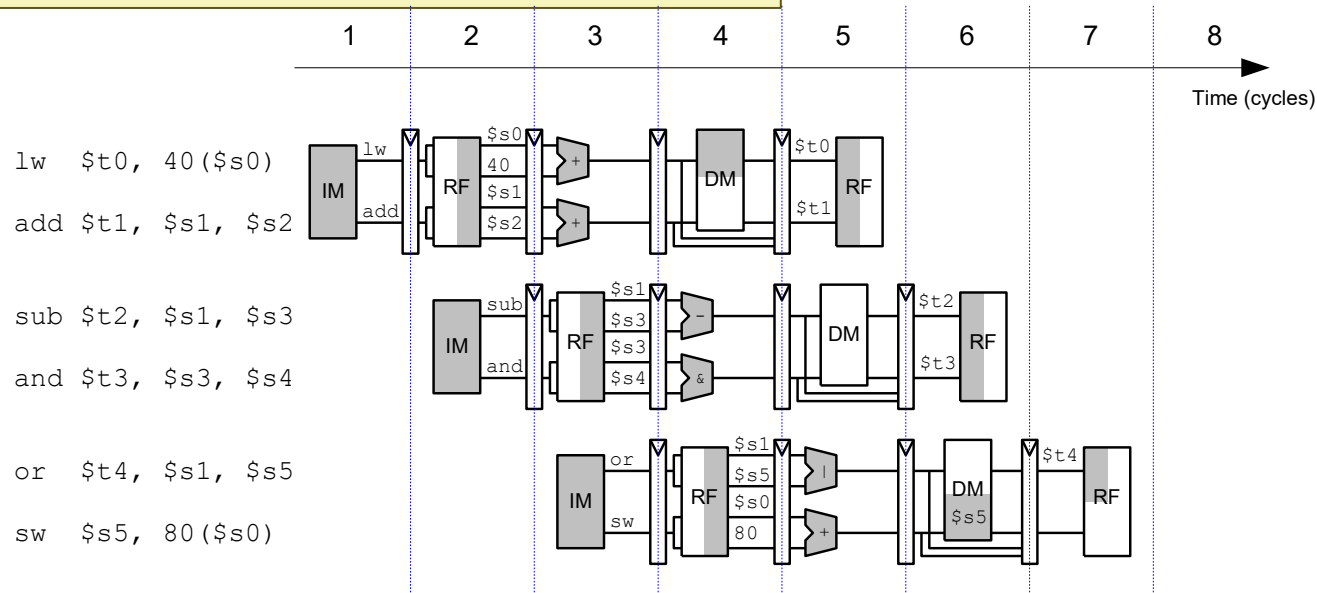


Here: Ideal IPC = 2

In-Order Superscalar Performance Example

```
lw  $t0, 40($s0)
add $t1, $s1, $s2
sub $t2, $s1, $s3
and $t3, $s3, $s4
or  $t4, $s1, $s5
sw  $s5, 80($s0)
```

Ideal IPC = 2



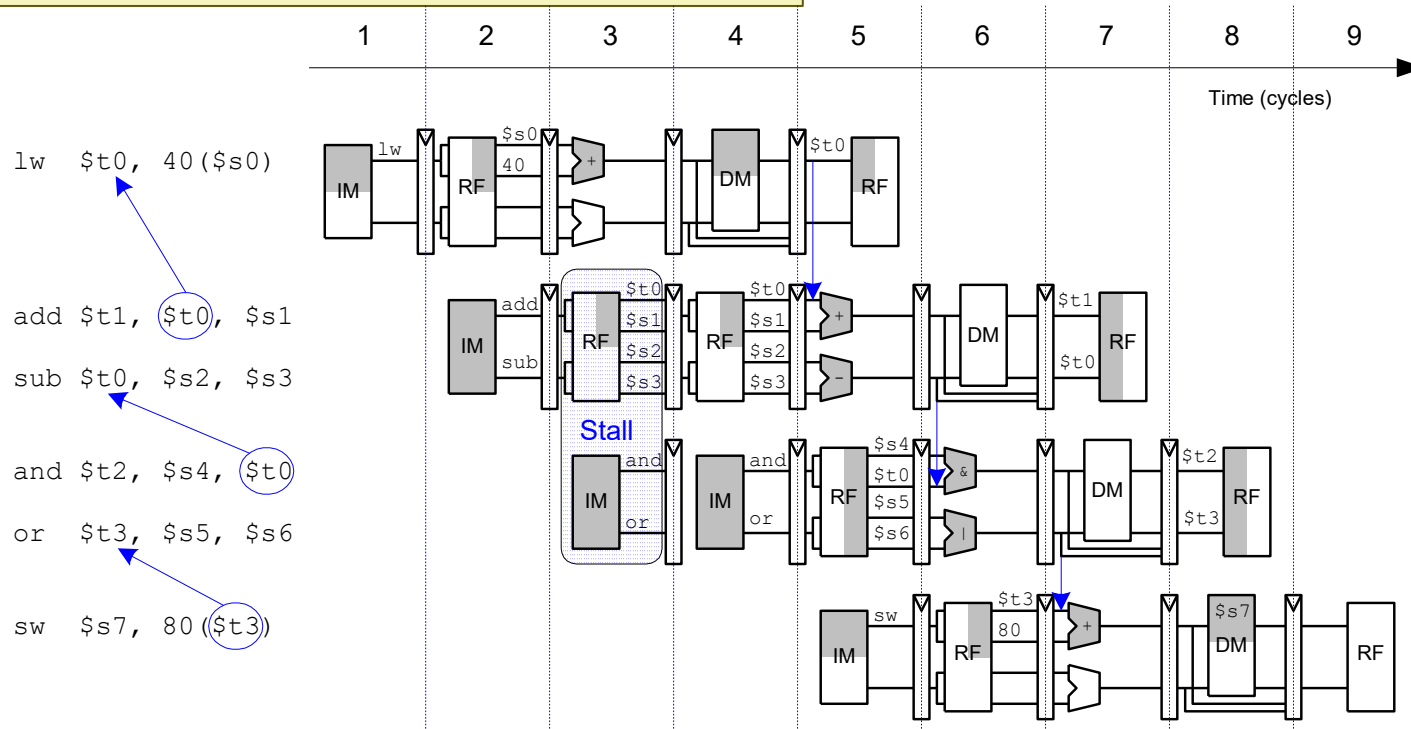
Actual IPC = 2 (6 instructions issued in 3 cycles)

Superscalar Performance with Dependences

```
lw  $t0, 40($s0)
add $t1, $t0, $s1
sub $t0, $s2, $s3
and $t2, $s4, $t0
or  $t3, $s5, $s6
sw  $s7, 80($t3)
```

Ideal IPC = 2

Can you reorder the instructions to get IPC = 2?



Actual IPC = 1.2 (6 instructions issued in 5 cycles)

Review: How to Handle Data Dependences

- Six fundamental ways of handling flow dependences
 - Detect and wait until value is available in register file
 - Detect and forward/bypass data to dependent instruction
 - Detect and eliminate the dependence at the software level
 - No need for the hardware to detect dependence
 - Detect and move it out of the way for independent instructions
 - Predict the needed value(s), execute “speculatively”, and verify
 - Do something else (fine-grained multithreading)
 - No need to detect

- Can employ all of these in superscalar processors

Superscalar Execution Tradeoffs

■ Advantages

- Higher instruction throughput
 - Higher IPC: instructions per cycle (i.e., lower CPI)

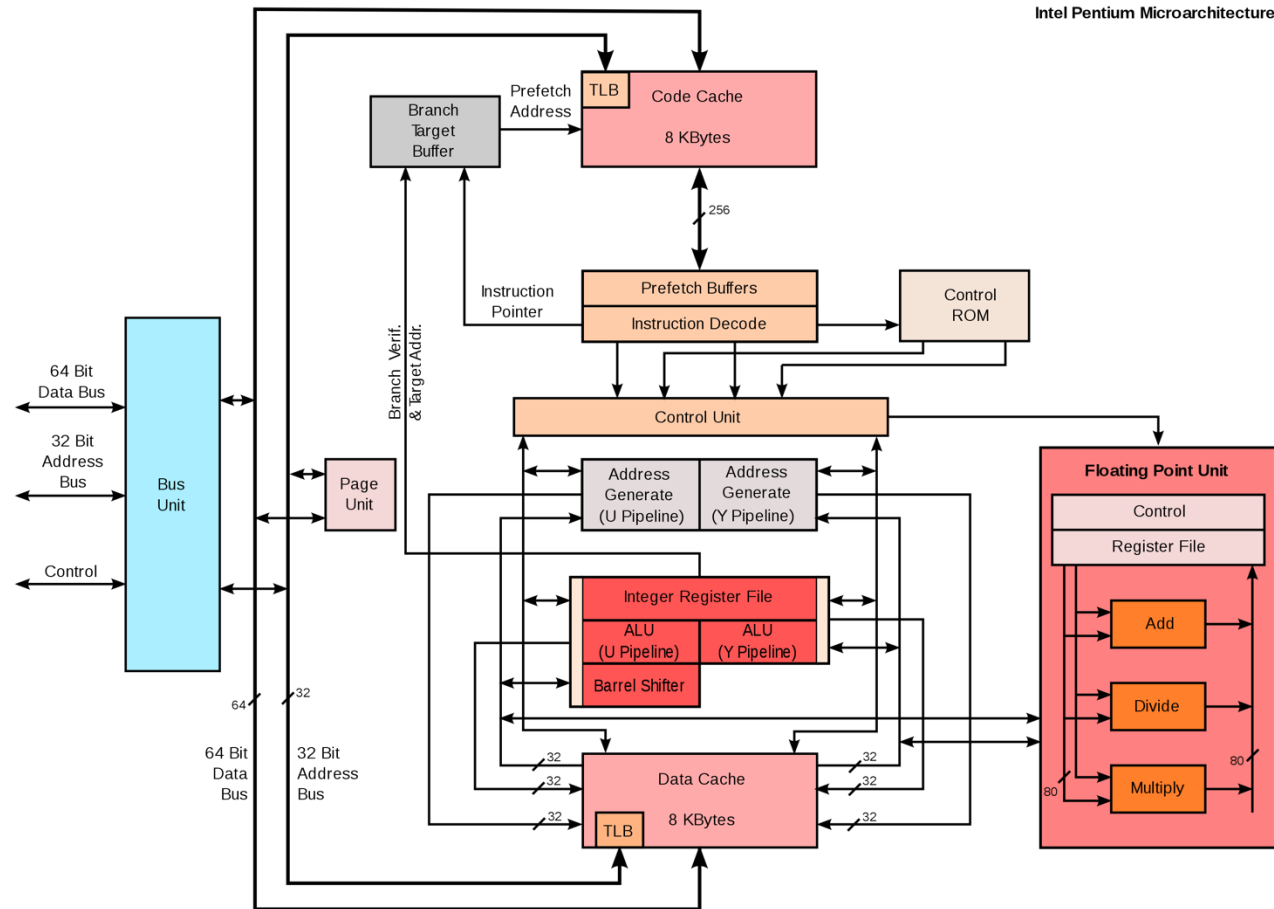
■ Disadvantages

- Higher complexity for dependence checking
 - Requires dependence checking between concurrent instructions
 - Register renaming becomes more complex in an OoO processor
 - Potentially lengthens critical path delay → clock cycle time
- More hardware resources needed

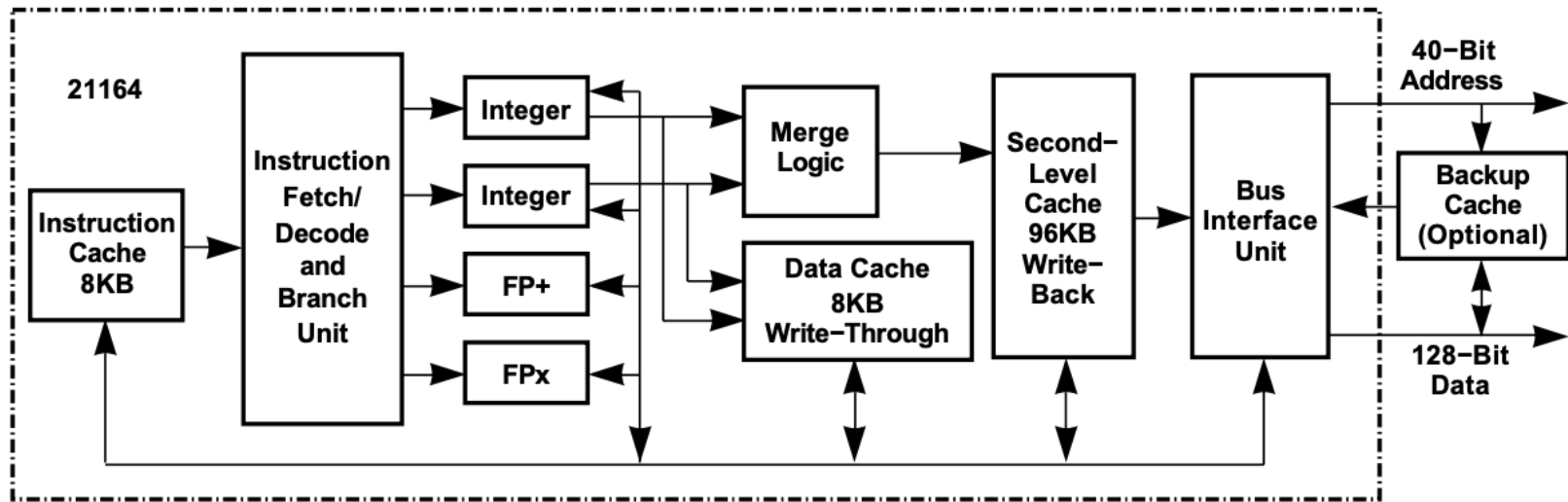
■ Recall: Execution time of an entire program

- **{# of instructions} x {Average CPI} x {clock cycle time}**

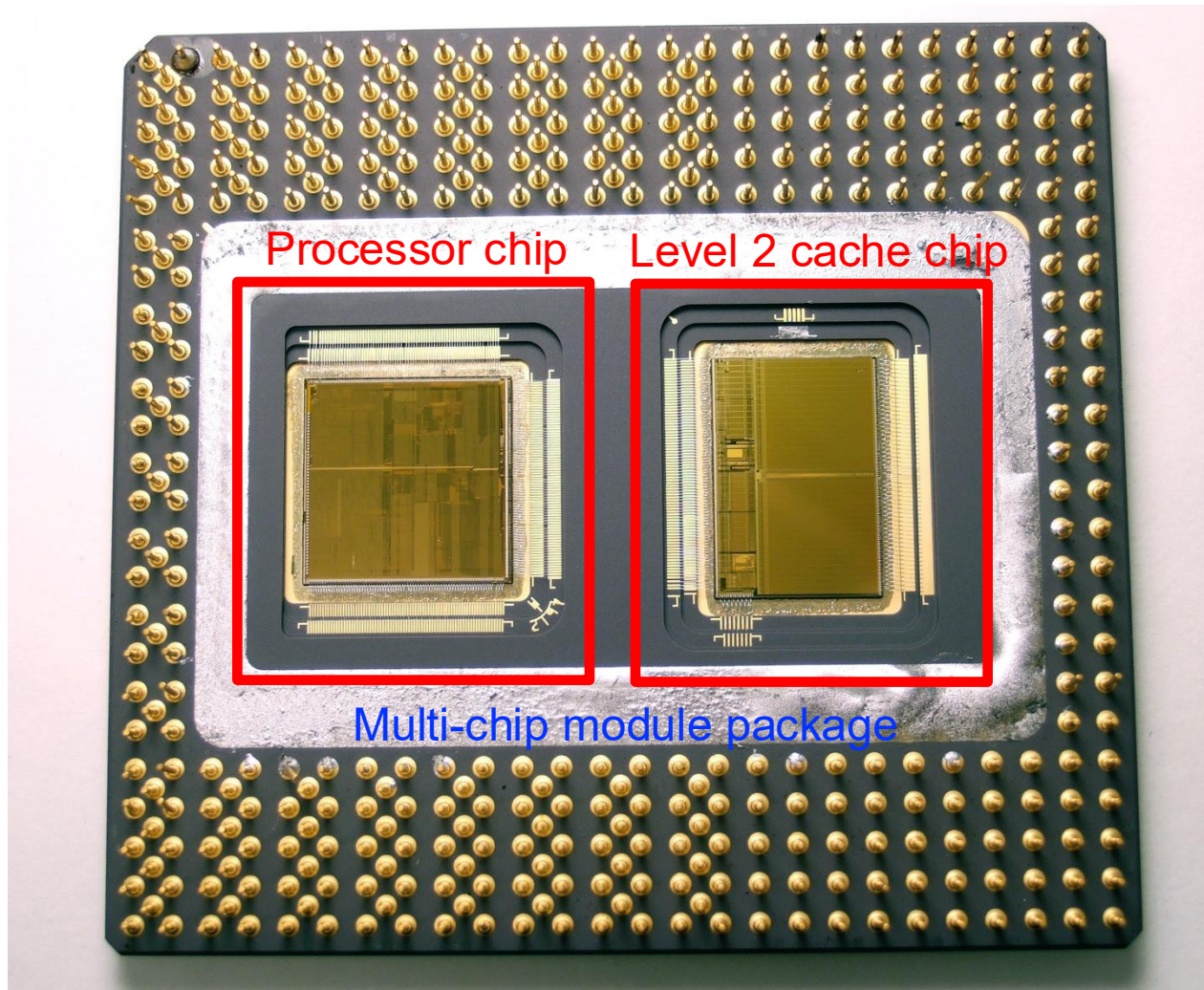
Intel Pentium: In-Order Superscalar (1993)



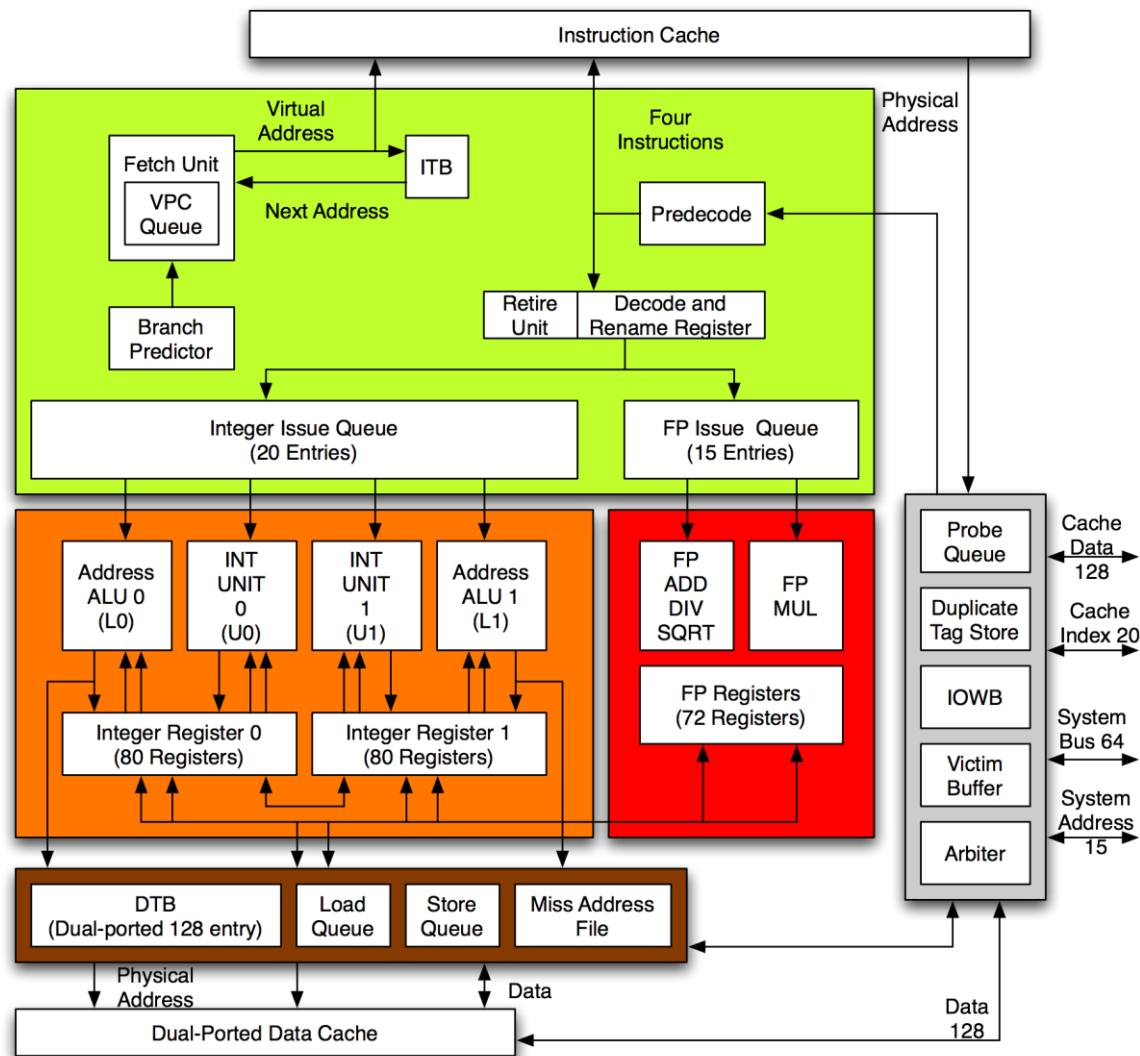
Alpha 21164: In-Order, Superscalar (1995)



Intel Pentium Pro: OoO, Superscalar (1995)



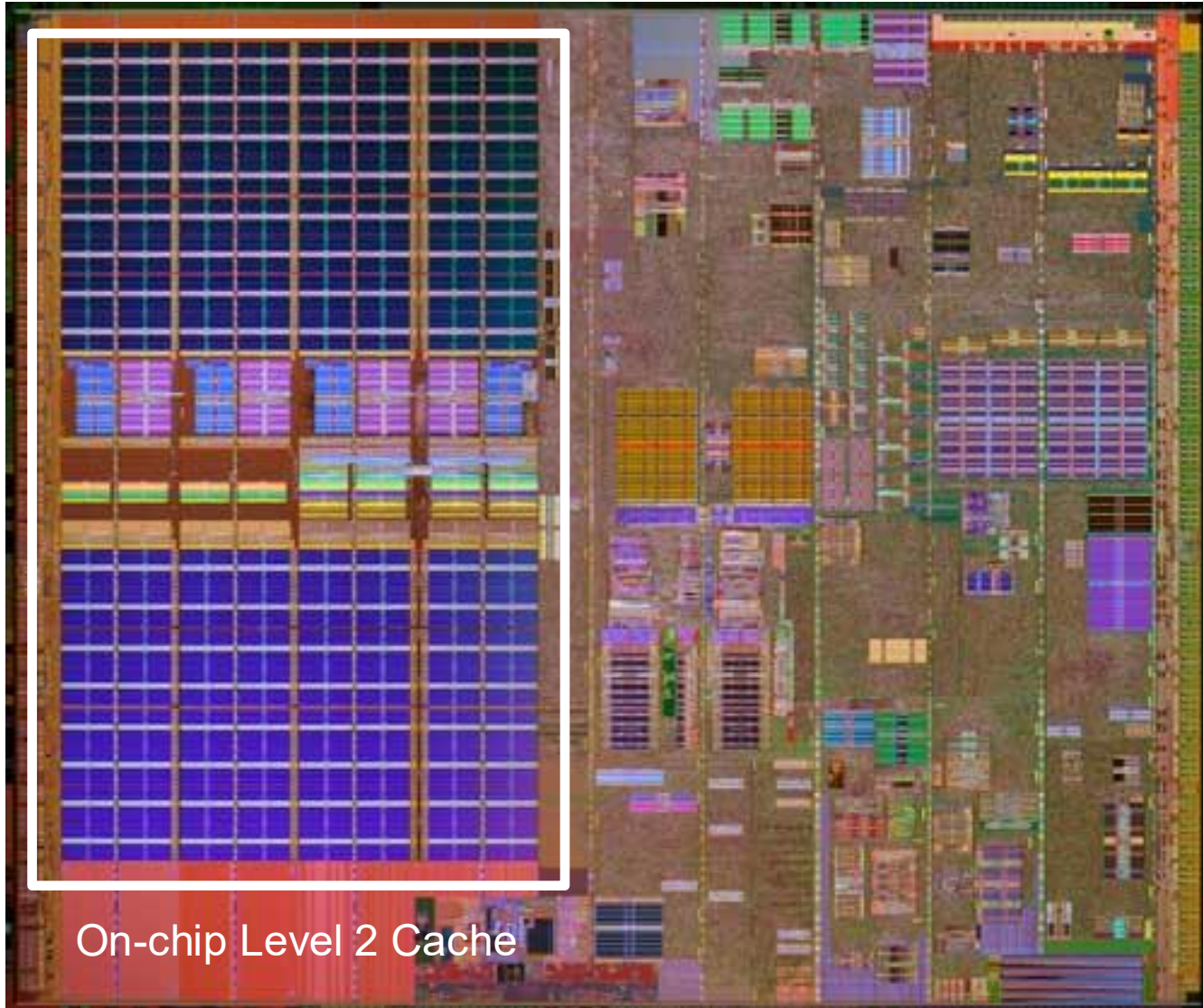
Alpha 21264: OoO, Superscalar (1998)



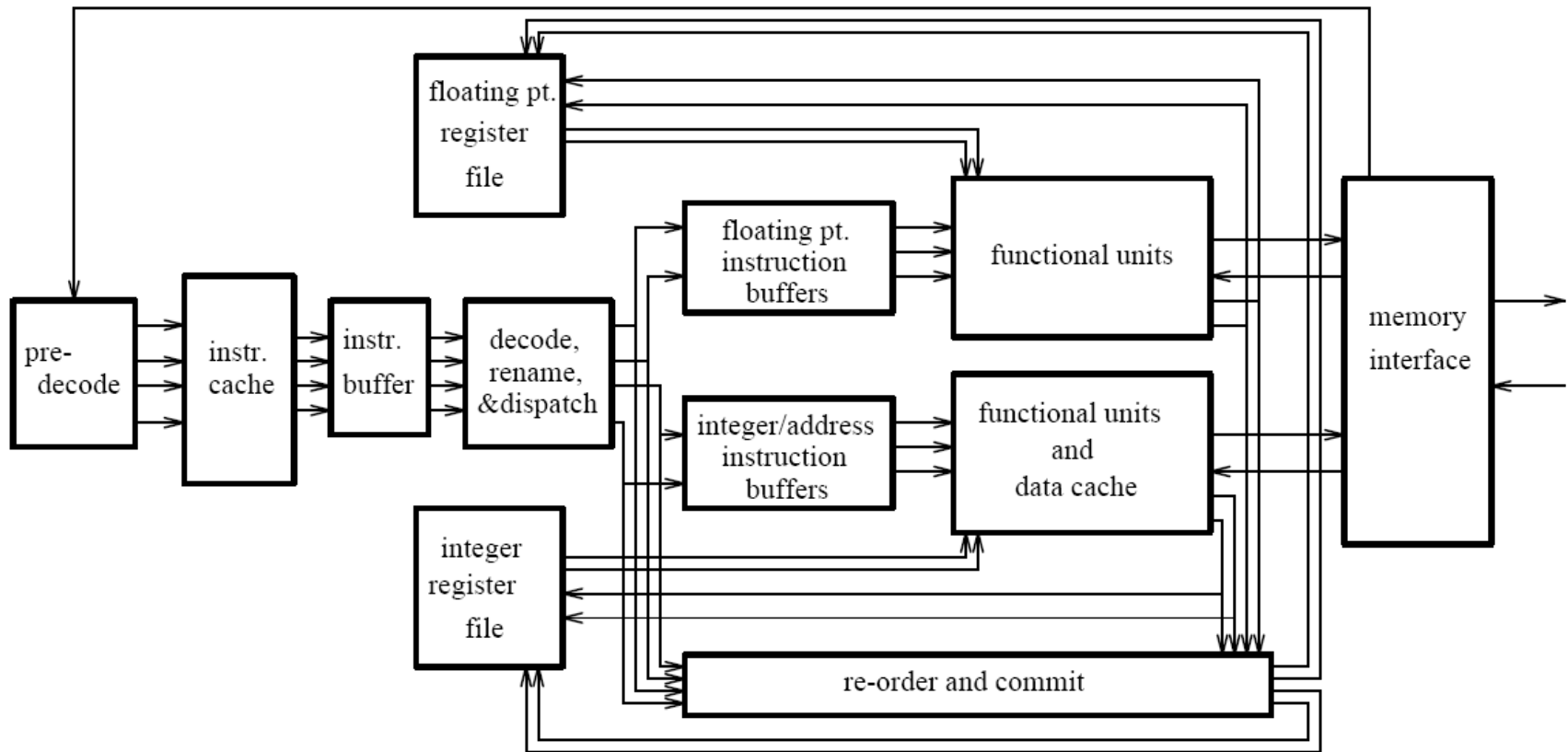
By Hellisp - Own work, Public Domain, <https://commons.wikimedia.org/w/index.php?curid=1261398>

Kessler, “[The Alpha 21264 Microprocessor](#),” IEEE Micro, March-April 1999.

Intel Pentium 4: OoO, Superscalar (2000)

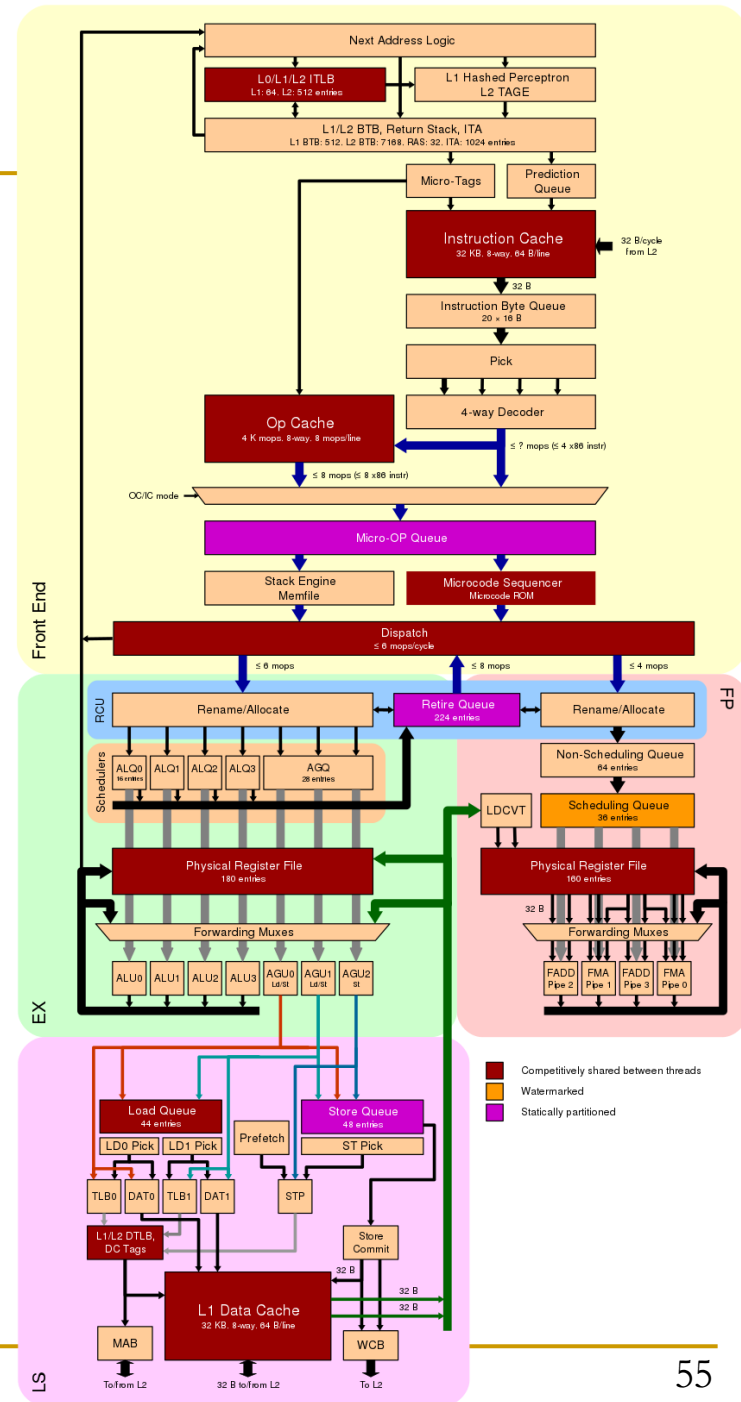
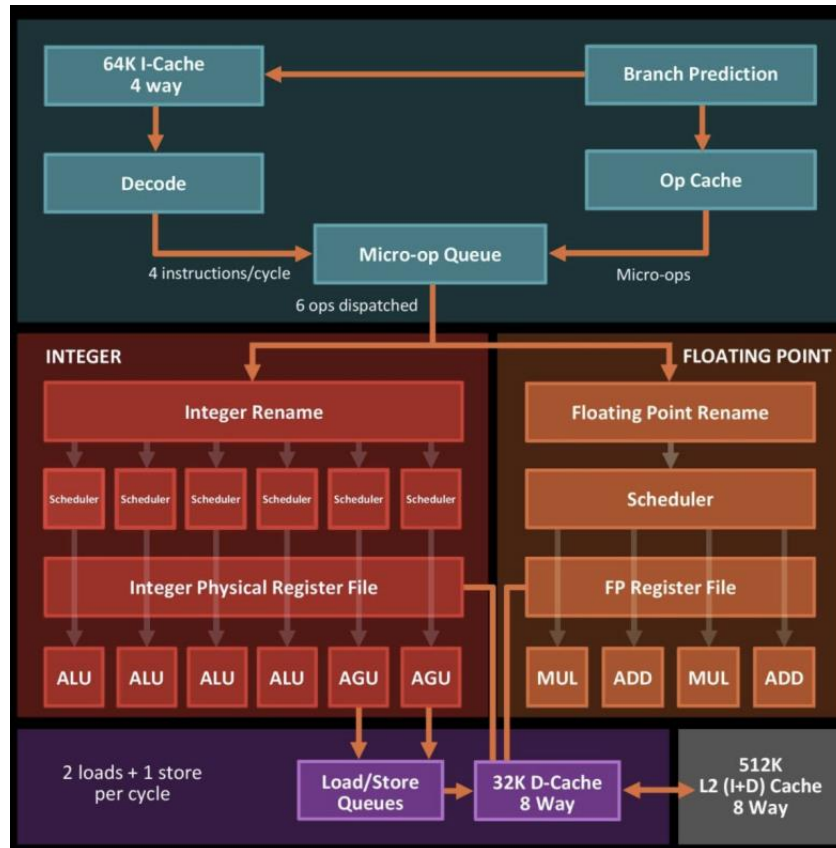


General Organization of a Superscalar+OoO Processor

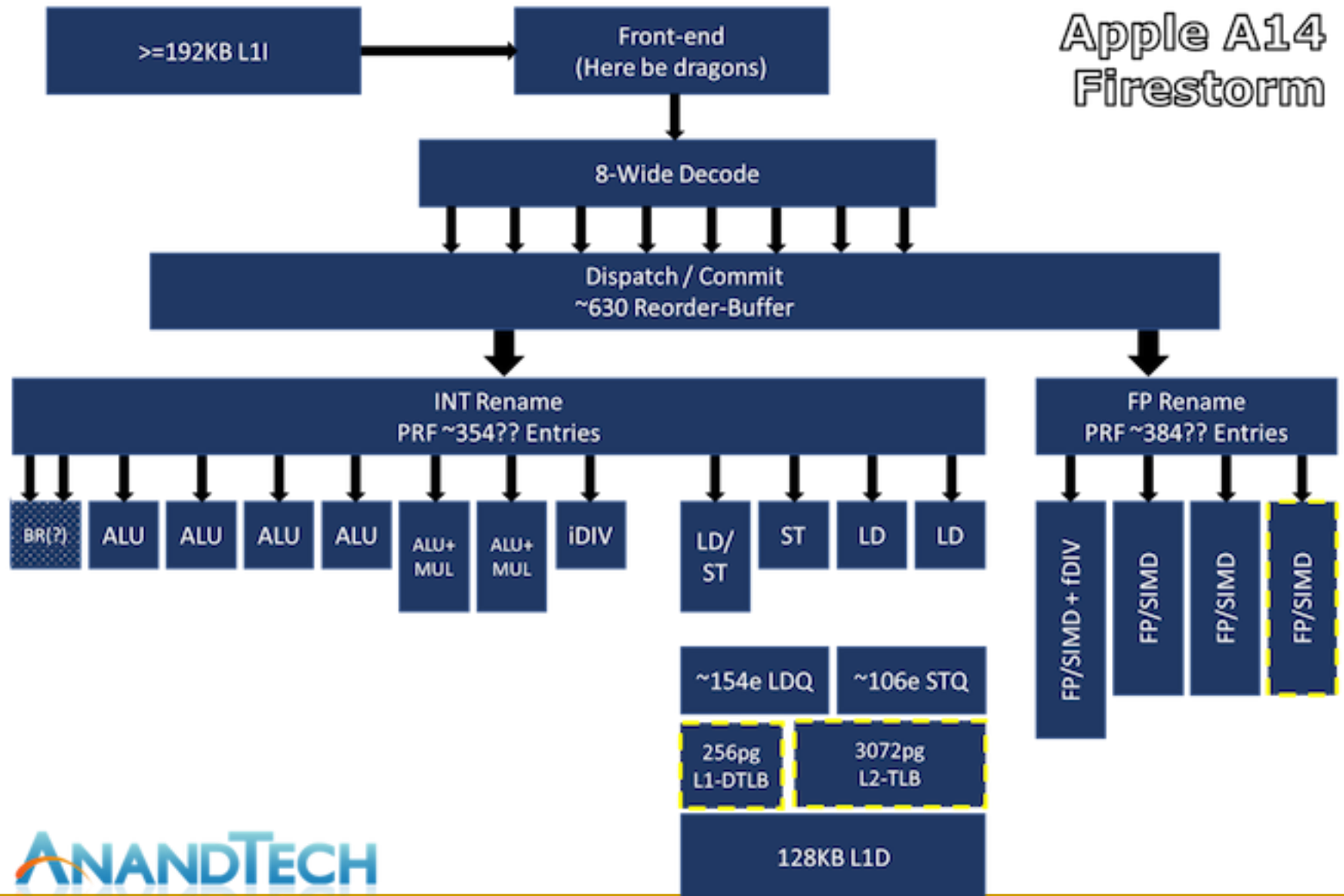


- Smith and Sohi, “**The Microarchitecture of Superscalar Processors**,” Proc. IEEE, Dec. 1995.

AMD Zen/Zen2? (2019)



Apple M1 Firestorm? (2020)



Digital Design & Computer Arch.

Lecture 15a: Dataflow & Superscalar Execution

Dr. Rahul Bera
Prof. Onur Mutlu

ETH Zürich
Spring 2026
17 April 2026